RELAYSPEC: RETARGETING FROZEN SPECULATIVE DRAFTERS WITH SMALL-DATA LINEAR INTERFACES

Anonymous authors
Paper under double-blind review

ABSTRACT

Feature-conditioned speculative drafters depend on the hidden states of the model used to train them. Reusing such a drafter with a larger target can therefore require running its source model alongside that target. RelaySpec replaces this dependency with a learned linear interface while keeping the target and drafter frozen. Across DFlash and EAGLE-3 drafters retargeted from Qwen3-4B to 8B and 14B, the original 4,096-record fits achieve 2.35–5.11 times autoregressive throughput on MATH-500. Separate EAGLE-3 measurements reduce peak allocated memory by 7.63–7.80 GiB relative to source reuse. Controlled scaling studies show that the interface can be learned from substantially fewer records: frozen 512-example maps retain 97.5% (DFlash) and 98.9% (EAGLE-3) of their 2,048-example maps' throughput on 256 held-out GSM8K questions with 8B targets. Both confidence intervals exceed 95% retention, although tight accuracy noninferiority remains unresolved. The simple dense map has the highest mapper throughput in the completed 8B long-output capacity comparisons. Wider MLPs and longer fitting do not consistently improve decoding. Under matched warm-training budgets, feature regression outperforms the tested connector-CE and LoRA alternatives. These results identify a low-cost reuse regime and its limits, including task-dependent slowdowns relative to source reuse, optimization sensitivity and finite-precision output differences.

1 INTRODUCTION

A developer may have a DFlash drafter trained for Qwen3-4B but want to serve Qwen3-8B or Qwen3-14B. The new target can check its proposals, but supplies different hidden states. This creates a deployment choice: train a new target-specific drafter, or reuse the existing drafter while paying for its source model to provide compatible features. RelaySpec addresses deployments that seek to reuse this training investment with limited additional fitting work. It learns a replacement for the source conditioning path, allowing the new target to supply the drafter's input. The practical goal is faster generation than plain AR while retaining much of a native target-specific drafter's throughput.

Speculative decoding accelerates autoregressive (AR) generation by checking several proposed tokens in one target pass (Leviathan et al., 2023). DFlash and EAGLE-3 improve proposals using features computed inside a language model (Chen et al., 2026; Li et al., 2025). We call the model that provided these features during drafter training the *source*. The *target* is the model whose token choices govern deployment. Running the source alongside a new target supplies compatible features, but also retains its transformer computation and memory.

RelaySpec runs the source and new target on shared text during fitting. It learns a linear map from the target's hidden states to the context that the source supplies to the drafter. At inference, the target already computes these states when checking proposals. Applying the map supplies the next drafting input without another source-transformer pass. The existing target verifier still decides which tokens to commit.

Only the map is trained. The target, drafter, inherited embedding and vocabulary projection stay fixed. The practical value is retaining an existing drafter's training investment when upgrading the verifier, while eliminating the source transformer from each generation cycle. A small interface is useful only

if it preserves enough proposal quality to make that removal worthwhile. Our experiments test this directly.

Our contribution has three parts. First, we identify and replace the source-conditioning boundary in two released drafter families, obtaining 2.35–5.11 times AR throughput on MATH-500 and separately saving 7.63–7.80 GiB of allocated memory. Second, frozen-checkpoint confirmation shows that 512 fitting examples retain 97.5% and 98.9% of the 2,048-example maps' speed. Third, we test whether simplicity sacrifices performance: matched linear/MLP capacity sweeps, longer fits, regularization and measured-budget CE/LoRA comparisons make the dense linear interface a strong empirical choice. It achieves the highest mapper throughput in the completed 8B long-output capacity comparisons, and feature regression wins every tested adaptation budget. The insight is that a frozen drafter's useful prediction behavior can survive a cheap change of conditioning interface. Increasing the expressiveness or training work of that interface need not improve decoding. Layer ablations identify a compact late-layer interface, and native-interface compression shows that this analysis also applies without retargeting. Cross-family, domain-composition and 14B experiments identify the limits of these findings.

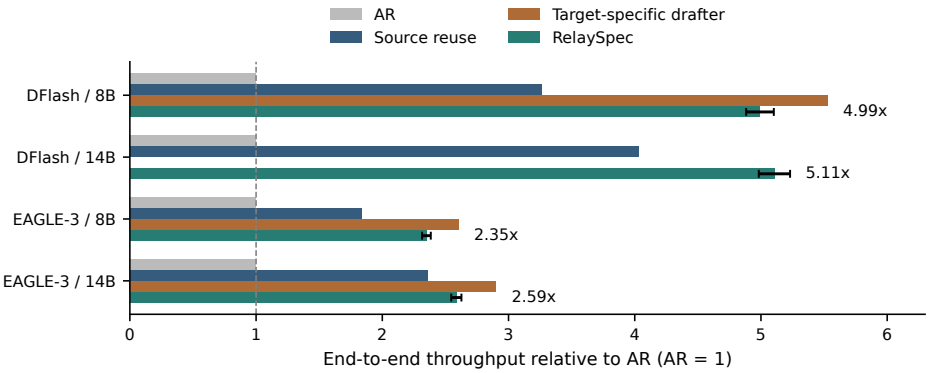

Figure 1: MATH-500 throughput relative to plain AR, measured within each paired run. RelaySpec reuses the 4B drafter. The native control uses a drafter released for the evaluated target. Whiskers show RelaySpec's 95% paired intervals. Native controls show the three evaluated releases. Each runtime uses one request per GPU.

2 Related work and positioning

Retargeting an inherited drafter. DFlash and EAGLE-3 condition their proposals on target features (Chen et al., 2026; Li et al., 2025), tying an inherited drafter to its training interface. TriSpec already supports adapter-only finetuning into a fixed EAGLE-family drafter, mapping proxy features to the expected feature space (Jiang et al., 2026). Adapter-only tuning is therefore not our novelty claim. TriSpec uses a proxy to approve some tokens without full-target verification. RelaySpec instead removes the source transformer from generation, retargets its inherited drafter to a larger verifier, and retains target-controlled acceptance. Our contribution is this deployment setting and its measured data, capacity and source-removal behavior across two drafting families.

Target-independent drafting and dynamic steering. PARD adapts an autoregressive draft model for parallel token prediction and reuses it across a target family (An et al., 2026). Thus reuse across targets is established prior work. Its target-independent drafter offers zero additional target-specific fitting at deployment. RelaySpec instead makes a small target-specific investment to reuse a feature-conditioned checkpoint and exploit the new target's internal states. SD^2 is a closer alignment alternative: it computes steering from verifier states and injects it into a pretrained drafter (Berdoz et al., 2026). Our fitted input map replaces source conditioning, whereas SD^2 learns steering within the drafter. The experiments include its public frozen-drafter configuration and PARD's released checkpoint, with their own matched AR controls (Appendix D). These comparisons expose practical

alternatives without attributing differences in runtime or inherited models to the alignment mechanism alone.

Distillation and drafter adaptation. OmniDraft uses online hybrid distillation and an n-gram cache to address cross-vocabulary deployment (Ramakrishnan et al., 2025). RelaySpec studies offline fitting with a shared vocabulary, so it does not replace heterogeneous-vocabulary verification (Timor et al., 2025a). RepSpec changes draft training through structural re-parameterization (Huo et al., 2026). VSD optimizes sequence acceptance, and the Draft-OPD preprint studies on-policy teacher feedback (Zou et al., 2026; Lei et al., 2026). These approaches improve a drafter's learned prediction behavior. RelaySpec holds that behavior fixed and regresses its input context. Our connector-CE and rank-32 LoRA experiments test whether updating token prediction is a better use of the same measured warm-training budget. Feature fitting wins every tested budget comparison (Table 4), which supports this inexpensive adaptation recipe without ruling out other distillation objectives or longer full-drafter training.

Frozen interfaces and the meaning of feature alignment. Adapters, low-rank updates and model stitching already adapt or connect frozen networks (Houlsby et al., 2019; Hu et al., 2022; Bansal et al., 2021). A linear connector alone is therefore not the novelty. The contribution is locating an existing speculative decoder's source-conditioning boundary, learning its replacement while preserving the target and drafter, and establishing the data, capacity and deployment consequences. Successful stitching need not imply equivalent information (Smith et al., 2025). We consequently test decoded speed and quality, equal-capacity linear and nonlinear maps, saved fitting trajectories and frozen-checkpoint confirmation. The observed gaps between training error, validation error and decoding speed are central findings rather than evidence that representation similarity is sufficient.

Proposal structure and execution are separate axes. Medusa changes prediction heads, EAGLE-2 and OPT-Tree allocate draft trees, and SpecInfer organizes tree verification (Cai et al., 2024; Li et al., 2024b; Wang et al., 2025a; Miao et al., 2024). Self-speculative methods reuse parts of the target itself (Zhang et al., 2024; Elhoushi et al., 2024), while speculative speculative decoding overlaps drafting and verification (Kumar et al., 2026). RelaySpec changes who supplies the drafter's input, not these execution policies. Its measured advantage is amortizing an inherited drafter with little fitting data and lower source-conditioning cost. We establish this on two drafter families and two target sizes, then identify when extra mapper capacity, optimization or broader training data helps and when it does not.

3 PRELIMINARIES

3.1 DRAFTING AND TARGET VERIFICATION

An autoregressive model generates one token at a time. Blockwise and speculative methods propose several tokens, then check them in a target pass (Stern et al., 2018; Xia et al., 2023; Leviathan et al., 2023). In greedy decoding, the verifier commits the longest matching prefix and a target-selected next token. RelaySpec preserves this decoder and changes the features supplied to its drafter.

3.2 THROUGHPUT AND ACCEPTED PROGRESS

Our primary metric is end-to-end output throughput: total generated tokens divided by total request time, including prompt processing. Let T_A, T_R and T_N denote this throughput for plain AR, RelaySpec and the native target-specific drafter. We report

$$\text{AR speedup} = T_R/T_A, \qquad \text{native throughput retained} = 100\,T_R/T_N\ \%. \tag{1}$$

Throughput includes prompt processing. We also report summed AR time relative to summed relay time because output lengths can differ. Accepted progress counts tokens advanced per verification cycle, including the target-supplied token. Removing source computation saves cycle time but can reduce accepted progress. Appendix E specifies this tradeoff and the position-wise acceptance convention.

4 METHOD

4.1 FIT THE INPUT CONSUMED BY THE FROZEN DRAFTER

We run the source and new target on the same text and collect hidden states after five selected transformer blocks. Concatenating these states gives the target vector h_t and source vector s_t at position t. The drafter's frozen projection F converts source features to its input width. We learn a bias-free matrix W that supplies this input from h_t. Figure 2 separates fitting from generation.

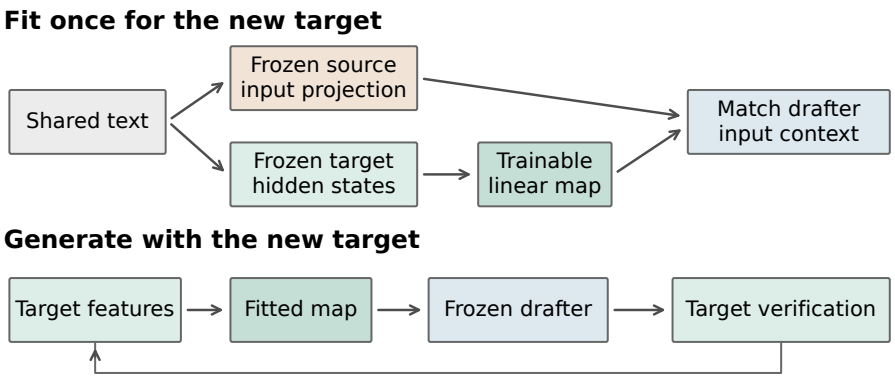

Figure 2: Fit once, then reuse the frozen drafter. The source and target process identical text during fitting. Only the map is updated. During generation, target features from committed positions feed the next draft.

DFlash consumes normalized projected features, while EAGLE-3 consumes the raw projection. Let N_{in} be fixed RMS normalization (Zhang & Sennrich, 2019) and N_{out} the released output normalization. Teacher and mapped contexts are

$$\begin{aligned} \text{DFlash:} \quad & c_t = N_{\text{out}}(Fs_t), \quad \widehat{c}_t = N_{\text{out}}(WN_{\text{in}}(h_t)), \\ \text{EAGLE-3:} \quad & c_t = Fs_t, \qquad\quad \widehat{c}_t = Wh_t. \end{aligned} \tag{2}$$

We fit W by minimizing the squared context error relative to the teacher's squared magnitude. For a record with n token positions,

$$\mathcal{L}(W) = \frac{1}{n} \sum_{t=1}^{n} \frac{\|\widehat{c}_t - c_t\|_2^2}{\max(\|c_t\|_2^2, \epsilon)}. \tag{3}$$

We average token losses within records and records across workers, with ϵ the smallest positive normal FP32 value. Teacher-magnitude normalization prevents larger vectors from dominating. The normalized DFlash objective differs from raw linear least squares. The original fits use 4,096 math problem-and-solution conversations, capped at 192 tokens. Supervision is the source context vector. Only W is updated, with 52.43M/65.54M weights for 8B/14B targets. Appendix A gives dimensions, layers and fitting settings.

4.2 DRAFT WITH THE NEW TARGET'S FEATURES

The target caches prompt states, and the map supplies the frozen drafter's context. The target verifies proposals in parallel, commits the matching prefix plus its next token, and crops the cache to the committed prefix. Mapped states feed the next cycle (Figure 6).

The standard greedy equivalence argument requires block verification to return the same token choices as single-token evaluation of each identical prefix (Leviathan et al., 2023). Finite-precision implementations can change those choices. We therefore measure task accuracy and token agreement separately from throughput. The map itself never supplies the scores used for target verification.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Models and runtime. We retarget the released Qwen3-4B DFlash and DeepSpec EAGLE-3 drafters to Qwen3-8B and Qwen3-14B (Yang et al., 2025; DeepSeek-AI, 2026). DFlash uses block length 16. EAGLE-3 uses the supported chain procedure with seven draft positions. Native controls use released target-specific drafters within the same implementation and proposal setting. The main AR comparisons use four NVIDIA L40S GPUs, with one sequential request per worker. Historical block-size and isolated-memory studies use RTX 6000 Ada GPUs. We compare methods within runs on the same hardware. Serving systems such as vLLM and SGLang also optimize cache use and request execution (Kwon et al., 2023; Zheng et al., 2024). We measure one request per worker. Table 6 gives the full protocol.

Workloads and data use. We evaluate math, code and conversation workloads because speculative speed varies across tasks (Xia et al., 2024; Abramovich et al., 2026). MATH-500 supplies 500 questions (Lightman et al., 2024). The breadth suite contains 128 GSM8K questions, 164 HumanEval tasks, 378 MBPP tasks and 80 two-turn MT-Bench conversations (Hendrycks et al., 2021; Cobbe et al., 2021; Chen et al., 2021; Austin et al., 2021; Zheng et al., 2023). The 32-question development set informed interface and loss choices. Historical evaluations retain this exposure history. Fitting and evaluation problem texts have zero normalized exact matches. Appendix H quantifies related templates using token overlap. All completed workload cells in each reported suite are included.

Controlled scaling protocol. The original 4,096-record checkpoints remain fixed for the broad workload results. A fixed-1,024-update sweep varies distinct records from 64 to 4,096. A separate NuminaMath study uses nested 16–32,768-record sets, 1,024 feature-validation records and 8,192 batch-four updates per fit. Frozen feature caches are shared across matched linear and MLP fits. The longer-output capacity comparisons use 128 exposed MATH questions and a 2,048-token cap. The 14B capacity and adaptation screens use sixteen exposed questions and a 256-token cap. These studies have separate checkpoints and are not pooled with the broad evaluation.

Quality and pairing. We score the same math outputs used for the AR timing comparison with the pinned Qwen math scorer. Historical code comparisons use EvalPlus base and expanded tests (Liu et al., 2023). MT-Bench measures two-turn generation time, with each method conditioning its second turn on its own first answer. Its bootstrap unit is the conversation. Other confidence intervals resample matched requests and condition on the fitted checkpoint. Model loading and external task scoring sit outside the request timer.

5.2 ACCELERATION OVER PLAIN AUTOREGRESSIVE DECODING

Table 1 reports absolute end-to-end throughput and AR-relative speed. DFlash RelaySpec reaches 186.49 tokens/s at 8B and 116.88 at 14B, compared with AR at 37.37 and 22.88. EAGLE-3 RelaySpec reaches 84.92 and 59.18 tokens/s, compared with 36.15 and 22.88 for AR. The resulting throughput ratios are 4.99, 5.11, 2.35 and 2.59. Each paired interval is above one.

Table 1: MATH-500 end-to-end tokens/s, including prompt processing. Progress R is mean recorded relay progress per cycle. Ratios and intervals use the AR arm of that same run. Native-drafter comparisons are in Table 8.

Drafter	Target	AR tok/s	Source tok/s	Relay tok/s	Relay / AR [95% CI]	Progress R
DFlash	8B	37.37	121.81	186.49	4.99 [4.88, 5.10]	6.98
DFlash	14B	22.88	92.15	116.88	5.11 [4.98, 5.23]	6.49
EAGLE-3	8B	36.15	66.44	84.92	2.35 [2.32, 2.38]	4.33
EAGLE-3	14B	22.88	53.96	59.18	2.59 [2.55, 2.63]	4.01

DFlash uses parallel proposals and the EAGLE-3 path drafts sequentially. These released-model comparisons also use different library versions, so cross-family speed does not isolate architecture.

5.3 NATIVE RETENTION AND SOURCE REMOVAL

RelaySpec retains 90.3% of native DFlash-8B throughput, reaching 186.49 versus 206.58 tokens/s. EAGLE-3 retains 90.2% at 8B and 89.1% at 14B (Table 8). These compare released drafters, not matched original training costs. Retained progress is lower than retained throughput because per-cycle work also differs: DFlash-8B advances 6.98 tokens per relay cycle versus 7.89 for native drafting.

Against source reuse, the relay's throughput ratios are 1.531 and 1.268 for DFlash, and 1.278 and 1.097 for EAGLE-3, at 8B and 14B respectively. This comparison holds the inherited drafter fixed. Source computation is removed from each cycle, while retained target states supply the context at the scale tested in Section 5.7.

5.4 MATH ACCURACY AND OUTPUT AGREEMENT

Table 2 scores the exact outputs timed in Table 1. Observed AR-to-relay accuracy differences range from -0.2 to $+0.8$ percentage points, and each paired interval includes zero. These measured outcomes support the accuracy comparison without treating token equality as a substitute for task quality.

Table 2: MATH-500 accuracy in percent. Difference is RelaySpec minus AR in percentage points. Token matches count identical full output sequences.

Drafter	Target	AR score	Relay score	Difference [95% CI]	Token matches
DFlash	8B	75.8	76.6	+0.8 [-1.4, +3.0]	117/500
DFlash	14B	79.2	79.2	+0.0 [-2.2, +2.0]	113/500
EAGLE-3	8B	75.8	75.6	-0.2 [-2.4, +2.0]	135/500
EAGLE-3	14B	79.2	79.4	+0.2 [-1.8, +2.2]	106/500

BF16 AR and block verification can select different tokens on the same prefix. Eight selected DFlash traces show target-score changes across native drafting, source reuse and RelaySpec. A separate Qwen3-8B diagnostic yields 12/16 AR matches in BF16 and 16/16 in full FP32 (Appendix C). These diagnostics do not establish bitwise agreement generally.

5.5 SPEED ACROSS WORKLOADS

Figure 3 and Table 9 report the completed DFlash AR-paired breadth suite. RelaySpec is 1.85–3.69 times faster than AR across these eight cells. Code gains at 14B remain substantial relative to AR even where source reuse is faster than the relay. The useful comparison depends on the deployment choice: AR measures the acceleration obtained, while source reuse measures the benefit of removing its source.

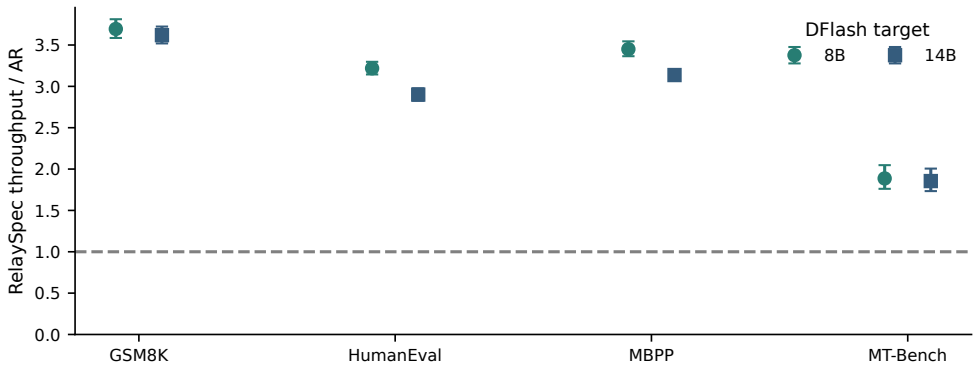

Figure 3: DFlash RelaySpec throughput relative to AR across workloads. Whiskers are paired 95% intervals. Chat intervals resample whole two-turn conversations. Full absolute values are in Table 9.

5.6 Fitting work and deployment memory

The selected maps have 52.43–65.54M weights. Fitting takes 85.6–118.5 seconds on four RTX 6000 Ada GPUs after model loading, including feature computation and optimization. Loading, export and deployment initialization are separate costs (Appendix I).

Separate EAGLE-3 processes measure memory after removing source layers. Peak allocated memory falls from 25.56 to 17.76 GiB at 8B and from 37.56 to 29.94 GiB at 14B. These are savings of 7.80 and 7.63 GiB. Source token embeddings and vocabulary projections required by a drafter remain part of deployment. The claim concerns the removed source transformer, not every tensor inherited from the source.

5.7 Ablation studies

5.7.1 How much data and mapper capacity are needed?

We separate distinct training examples from optimization work. With 8,192 batch-four updates held fixed, the NuminaMath sweep spans 16 to 32,768 distinct records. DFlash-8B throughput rises from 96.9 tokens/s at 16 examples to 192.8 at 512, then ranges from 196.9 to 198.4 at 2,048–32,768 (Figure 4). Thus 16 examples already give $2.58\times$ AR throughput, and 512 retain 97.2% of the observed best. All eleven mappers score 105/128. These are single-seed development results, not a general minimum-data threshold. Appendix C retains the separate historical MATH-only study.

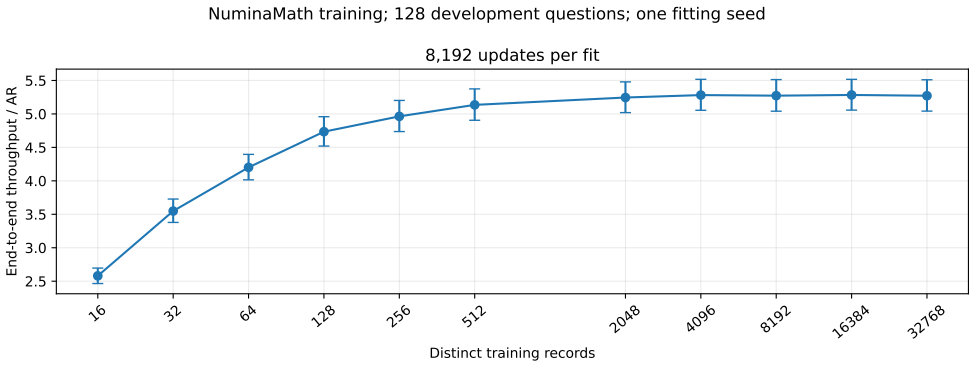

Figure 4: DFlash-8B data scaling on 128 development questions, with NuminaMath training and 8,192 updates per fit. All dataset sizes are measured with shared controls in one campaign. Paired 95% request intervals exclude fitting-seed variation and selection uncertainty.

The NuminaMath capacity study crosses 512/2,048 examples with dense, factored linear and GELU MLP maps over seven widths, using 8,192 updates per cell. This holds parameter count equal between each factored map and MLP. Validation error decreases with capacity and then flattens (Figure 5). On 128 questions at a 2,048-token cap, dense $N = 512$ retains 97.78% [97.09,98.47]% of $N = 2048$ throughput. Linear-1024 at $N = 512$ retains 95.00% [94.14, 95.85]% with 23.59M parameters instead of 52.43M. The latter interval crosses the 95% boundary. These are development comparisons with fixed fitting seeds.

Two-layer conditioning also transfers to native drafting. Frozen retargeted and native two-layer interfaces each cut projection parameters by 60%, retaining 97.63% [96.23,99.03]% and 97.05% [95.93,98.21]% throughput on disjoint 64-question GSM8K sets. Outputs match their respective full-interface references (Appendix C).

EAGLE-3 repeats the dense data-count contrast on the same development questions, retaining 98.28% [97.59, 99.01]%. Its matched MLP remains slower than dense. Table 38 retains all eight maps.

Confirmation with frozen checkpoints. Before generating confirmation outputs, we fix the dense checkpoints and analysis criteria, then evaluate 256 GSM8K questions excluded from the recorded

development history (Table 3). Both families exceed the predeclared 95% speed-retention threshold. The two data sizes have identical per-question correctness within each family. Relative to AR, conservative accuracy-difference intervals are $[-3.07, 3.07]$ points for DFlash and $[-1.63, 3.14]$ for EAGLE-3. Neither establishes the one-point noninferiority margin.

Table 3: Frozen small-data confirmation on 256 GSM8K questions. Retention is a percentage relative to the 2,048-example mapper in the same family, with paired 95% intervals. Outputs are capped at 2,048 tokens.

Family	Examples	Tokens/s	Retention (95% CI)	Correct	At cap
DFlash	512	157.8	97.5 [96.9,98.1]	236/256	0
DFlash	2,048	161.9	100 (reference)	236/256	0
EAGLE-3	512	100.0	98.9 [98.1,99.7]	238/256	0
EAGLE-3	2,048	101.1	100 (reference)	238/256	0

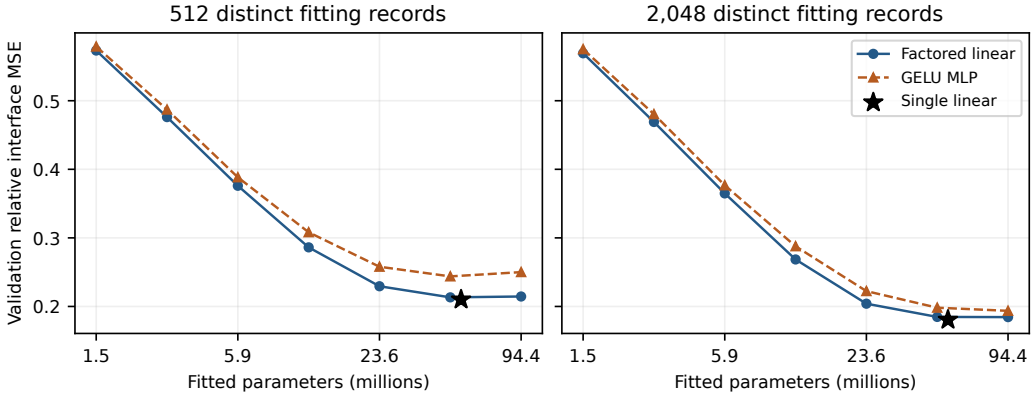

Figure 5: Complete primary capacity study at 8,192 updates per cell. Markers compare validation interface error, with one fitting seed. The curves do not establish decoding speed or answer quality.

Better feature fitting need not mean faster decoding. The 94.37M-parameter MLP-4096 at $N = 2048$ reaches 190.15 tokens/s, below dense. Extending the width-512 MLP from 8,192 to 32,768 updates on the same 2,048 records raises throughput only from 154.39 to 157.68. All tested mappers score 105/128 at this cap, versus AR's 104/128, which does not establish a one-percentage-point noninferiority margin. The regularization study finds no matched endpoint benefit from its L2 settings and only small weight-decay gains in feature validation. The dense linear map has the highest mapper throughput in both completed 8B long-output capacity comparisons. Thus its simplicity is supported by measured alternatives, including equal-capacity nonlinear maps, rather than assumed from parameter count. All outcomes remain in the appendix.

Replication and workload dependence. The complete 14B grids repeat the small-data/capacity comparison in both drafter families. DFlash's wide MLP fits its training features better than dense but has worse validation error and lower capped decoding throughput. EAGLE-3 shows late degradation in both dense 512-example seeds and a transient wide-linear loss spike. These results delimit the shared optimization recipe rather than prove a disadvantage of a function class (Appendix C.6). On the 8B capped long-output development set, dense $N = 512$ retains 97.4–97.9% of dense $N = 2048$ throughput across MATH difficulty groups. Very long inputs have too few examples for a reliable generalization claim.

Training composition changes the speed tradeoff. Replacing half of 2,048 math records with general instructions reduces dense general-domain feature error by 32.9% but raises math error by 10.2%. Code and chat throughput improve by 2.9% and 4.9%, while MATH throughput falls 1.2%. A second seed repeats this pattern. Matched arms generate identical tokens on all 160 development

requests. For two-layer maps, a separate 16-conversation extension gives a 7.5% dialogue gain at both fitting seeds, compared with 0.3–1.1% for native interface students (Appendix F).

5.7.2 Feature fitting wins the matched adaptation comparisons

Feature regression achieves the highest throughput at every tested warm budget (Table 4). The comparison fixes 512 training records, charges CE and LoRA for their initial mapper, and gives each an equal three-rate development screen. Across all three budgets, CE retains 73.2–76.6% and the two LoRA fits retain 79.4–82.7% of feature throughput. Every paired 95% interval lies below parity. This supports feature regression as the best tested adaptation under these budgets. The sixteen-question, 256-token screen does not establish full-answer quality or the ranking of other objectives. Cache preparation and export remain separate costs (Appendix C.7).

Table 4: Decoding tokens/s after matched warm-training budgets, using one common DFlash-8B runtime and the same 512 records. Columns are nominal training seconds. Bold marks the best tested adaptation in each column. All twelve fits and paired intervals appear in Table 42.

Adaptation	22.9 s	91.4 s	365.6 s
Feature MSE	**145.05**	**144.80**	**144.55**
Connector CE	111.04	109.60	105.79
LoRA32 (1729)	119.30	115.02	118.49
LoRA32 (1730)	119.52	115.49	119.55

Public-runtime baselines. Table 5 places the completed public baselines beside RelaySpec. The relay has the highest measured throughput, but the separate runtime controls prevent an algorithm-only ranking. Table 44 distinguishes their adaptation mechanisms.

Table 5: Public-runtime comparison on 128 MATH development questions, 2,048-token cap. Correct counts are out of 128, with each system's AR in parentheses. Precision, attention and libraries differ, so speedups use own-runtime AR. Settings and paired quality uncertainty appear in Appendix D.

System	Tok/s	Own AR tok/s	Speedup [95% CI]	Correct (AR)
RelaySpec ($N = 512$)	193.68	37.72	5.14 [4.90, 5.37]	105 (104)
PARD	105.63	31.67	3.34 [3.22, 3.45]	105 (103)
SD^2 (frozen drafter)	19.92	12.48	1.60 [1.56, 1.63]	103 (103)

6 Conclusion

RelaySpec makes an existing feature-conditioned drafter useful with a larger verifier by learning one replacement input map. Its value is the combination of retained drafting behavior, removed source-transformer cost and little additional fitting data. Dense linear mapping is a competitive choice after testing nonlinear capacity, longer optimization, regularization and token-prediction adaptation. The interface also exposes which target features sustain drafting: late-layer selection cuts mapper parameters by 60% while retaining 97.6% throughput in a fixed comparison. A native two-layer student retains 97.1% throughput with 60% fewer projection parameters in a separate fixed comparison. These findings motivate adapting and inspecting the inherited interface before investing in a new drafter.

These conclusions concern shared-vocabulary Qwen3 models, greedy decoding and one request per worker. Code workloads can favor source reuse, and EAGLE-3 optimization depends on the recipe. Frozen GSM8K confirmation supports speed retention but not tight accuracy noninferiority. Development exposure, possible shared math templates and limited fitting seeds restrict generalization. The 14B capacity and adaptation screens remain capped comparisons, and public-runtime baselines do not isolate an algorithm-only ranking.

AI USE STATEMENT

Generative AI assisted with implementation, code review, literature search, analysis and manuscript editing. Numerical tables and plots are generated from recorded artifacts with checked method, request and scorer identity. Scientific interpretation and submission decisions remain the authors' responsibility.

REPRODUCIBILITY STATEMENT

The appendix specifies model identifiers, feature layers, fitting settings, measurement boundaries, complete workload results and data checks. The accompanying implementation includes configurations, request records, scorer provenance and scripts that regenerate the reported assets.

REFERENCES

Talor Abramovich, Maor Ashkenazi, Izzy Putterman, Benjamin Chislett, Tiyasa Mitra, Bita Darvish Rouhani, Ran Zilberstein, and Yonatan Geifman. SPEED-Bench: A unified and diverse benchmark for speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2604.09557. Accepted paper.

Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost PARallel draft model adaptation. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=XbOyv7iVGL.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL https://arxiv.org/abs/2108.07732.

Yamini Bansal, Preetum Nakkiran, and Boaz Barak. Revisiting model stitching to compare neural representations. In *Advances in Neural Information Processing Systems 34*, 2021. URL https://papers.nips.cc/paper/2021/hash/01ded4259d101feb739b06c399e9cd9c-Abstract.html.

Frédéric Berdoz, Peer Rheinboldt, and Roger Wattenhofer. Steering pretrained drafters during speculative decoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 30067–30075, 2026. doi: 10.1609/aaai.v40i36.40255. URL https://ojs.aaai.org/index.php/AAAI/article/view/40255.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://proceedings.mlr.press/v235/cai24b.html.

Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2602.06036. Accepted paper.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

DeepSeek-AI. DeepSpec: A full-stack codebase for training and evaluating speculative decoding draft models. Official open-source software, 2026. URL https://github.com/deepseek-ai/DeepSpec.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. URL https://aclanthology.org/2024.acl-long.681/.

Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://proceedings.mlr.press/v235/fu24a.html.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems 34, Datasets and Benchmarks Track*, 2021. URL https://openreview.net/forum?id=7Bywt2mQsCe.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, 2019. URL https://proceedings.mlr.press/v97/houlsby19a.html.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022. URL https://www.microsoft.com/en-us/research/publication/lora-low-rank-adaptation-of-large-language-models/.

Shijing Hu, Jingyang Li, Xingyu Xie, Zhihui Lu, Kim-Chuan Toh, and Pan Zhou. GRIFFIN: Effective token alignment for faster speculative decoding. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/b6e67ae290635d0874c4cb43ba2a2cfb-Abstract-Conference.html.

Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. RepSpec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554c1f-Abstract-Conference.html.

Haoyun Jiang, Junqi He, Feng Hong, Xinlong Yang, Jianwei Zhang, Zheng Li, Zhengyang Zhuge, Zhiyong Chen, Bo Han, Junyang Lin, and Jiangchao Yao. TriSpec: Ternary speculative decoding via lightweight proxy verification. *arXiv preprint arXiv:2601.23180*, 2026. doi: 10.48550/arXiv.2601.23180. URL https://arxiv.org/abs/2601.23180.

Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/1b96f01343ff10150e6719eb163e1536-Abstract-Conference.html.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL https://doi.org/10.1145/3600006.3613165.

Haodi Lei, Yafu Li, Haoran Zhang, Shunkai Zhang, Qianjia Cheng, Xiaoye Qu, Ganqu Cui, Bowen Zhou, Ning Ding, Yun Luo, and Yu Cheng. Draft-OPD: On-policy distillation for speculative draft models. *arXiv preprint arXiv:2605.29343*, 2026. URL https://arxiv.org/abs/2605.29343.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, 2024a. URL https://proceedings.mlr.press/v235/li24bt.html.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024b. URL https://aclanthology.org/2024.emnlp-main.422/.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems 38*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/c7b5a35ea98b62512a869c19ea7b03cb-Abstract-Conference.html.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/aca97732e30bcf1303bc22ac3924fd16-Abstract-Conference.html.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024. URL https://arxiv.org/abs/2305.09781.

Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. OmniDraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3c2fe1417eed1c6ff9acf169617981ea-Abstract-Conference.html.

Damian Smith, Harvey Mannering, and Antonia Marcu. Functional alignment can mislead: Examining model stitching. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 55972–55998, 2025. URL https://proceedings.mlr.press/v267/smith25a.html.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise Parallel Decoding for Deep Autoregressive Models. In *Advances in Neural Information Processing Systems 31*, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/c4127b9194fe8562c64dc0f5bf2c93bc-Abstract.html.

Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Gaurav Jain, Oren Pereg, Moshe Wasserblat, and David Harel. Accelerating LLM inference with lossless speculative decoding algorithms for heterogeneous vocabularies. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025a. URL https://proceedings.mlr.press/v267/timor25a.html.

Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Oren Pereg, Moshe Wasserblat, Tomer Galanti, Michal Gordon-Kiwkowitz, and David Harel. Distributed speculative inference: Speculation parallelism for provably faster lossless language model inference. In *International Conference on Learning Representations*,

2025b. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/b36554b97da741b1c48c9de05c73993e-Abstract-Conference.html.

Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. OPT-Tree: Speculative Decoding with Adaptive Draft Tree Structure. *Transactions of the Association for Computational Linguistics*, 2025a. URL https://aclanthology.org/2025.tacl-1.8/.

Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. SVD-LLM: Truncation-aware singular value decomposition for large language model compression. In *International Conference on Learning Representations*, 2025b. URL https://arxiv.org/abs/2403.07378.

Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu Wei, and Zhifang Sui. Speculative Decoding: Exploiting Speculative Execution for Accelerating Seq2seq Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. URL https://aclanthology.org/2023.findings-emnlp.257/.

Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. URL https://aclanthology.org/2024.findings-acl.456/.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint arXiv:2506.09501*, 2025. URL https://arxiv.org/abs/2506.09501.

Zhihang Yuan, Yuzhang Shang, Yue Song, Dawei Yang, Qiang Wu, Yan Yan, and Guangyu Sun. ASVD: Activation-aware singular value decomposition for compressing large language models. *arXiv preprint arXiv:2312.05821*, 2023. URL https://arxiv.org/abs/2312.05821.

Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems 32*, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. URL https://aclanthology.org/2024.acl-long.607/.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 37*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

Xiandong Zou, Jianshu Li, Jing Huang, and Pan Zhou. Variational speculative decoding: Rethinking draft training from token likelihood to sequence acceptance. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL https://ink.library.smu.edu.sg/sis_research/11190/.

APPENDIX

The appendix gives implementation details (Appendix A), complete workload results (Appendix B), fitting studies (Appendix C), acceptance measurements (Appendix E), data checks (Appendix H) and fitting-time and memory measurements (Appendix I).

A IMPLEMENTATION AND MEASUREMENT DETAILS

Table 6: Evaluation protocol. Main fitting choices remain fixed across workloads. Full identifiers and runtime versions are in Appendix A.

Component	Setting
Primary baseline	Plain greedy AR with the same target, prompt and runtime
Other controls	Native target-specific drafter and optimized source reuse
Generation	BF16, SDPA attention, thinking disabled, maximum 2,048 new tokens
Relay fitting	4,096 records, 192-token cap, 1,024 AdamW updates (Loshchilov & Hutter, 2019), four records/update
Optimizer	Learning rate 6×10^{-4}, zero weight decay, gradient clipping 1.0
Timing	Synchronized request timer, warmup excluded, rotated method order
Uncertainty	10,000 paired resamples, whole conversations for two-turn chat

Released models. The source is `Qwen/Qwen3-4B`. Targets are `Qwen/Qwen3-8B` and `Qwen/Qwen3-14B`. The source DFlash is `z-lab/Qwen3-4B-DFlash-b16`, and its native 8B control is `z-lab/Qwen3-8B-DFlash-b16`. EAGLE-3 uses DeepSpec's `eagle3_qwen3_4b_ttt7`, `eagle3_qwen3_8b_ttt7` and `eagle3_qwen3_14b_ttt7` releases. All model and upstream revisions are fixed in the accompanying configurations.

Features and dimensions. Layer indices count transformer blocks from zero. Source and Qwen3-8B states come from blocks $[1, 9, 17, 25, 33]$. Qwen3-14B states come from $[1, 10, 19, 28, 37]$. Joining five 4,096-wide states gives a 20,480-wide 8B input. Joining five 5,120-wide states gives a 25,600-wide 14B input. Both drafter interfaces have width 2,560. The matrix dimensions are therefore $2,560 \times 20,480$ and $2,560 \times 25,600$. The DFlash input normalization has no trainable gain or bias. The released output normalization remains frozen.

Training. Four workers average record losses at each update. Seed 1729 fixes the selected fit. AdamW uses learning rate 0.0006, zero weight decay and gradient clipping at 1.0. Computation uses BF16, with the feature loss computed in FP32. Each of 1,024 updates processes four records, each capped at 192 rendered tokens. The same map remains fixed across tasks.

Runtime and timing. The main AR runs use Python 3.12.13 and PyTorch 2.9.1 with CUDA 12.8. DFlash uses Transformers 5.3.0. EAGLE-3 uses the pinned 5.10.2 overlay. Each worker owns a full target replica on one L40S GPU. Four workers process independent requests rather than splitting one model over GPUs. One warmup per method precedes measured requests. Method order rotates across requests. The synchronized timer includes prompt processing, drafting, verification, feature updates and runtime bookkeeping. The historical block and memory experiments use RTX 6000 Ada GPUs.

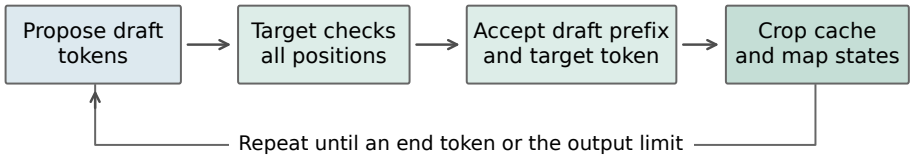

Figure 6: One generation cycle. RelaySpec changes the drafter's context. The target retains control over committed tokens and the shared decoder retains responsibility for positions, stopping and cache updates.

Stopping and conversation history. Generation stops at the first end token or the output cap. Capped requests remain in both timing and quality aggregates. Each MT-Bench method uses its own first-turn output as second-turn context. Two turns form one resampling unit. The breadth run has 750 initial records and 830 timed requests after expanding 80 conversations to two turns.

Aggregation. Each method's throughput is the sum of its output-token counts divided by the sum of its request times. Each of 10,000 resamples recomputes this ratio from matched requests, using bootstrap seed 1729. Confidence bounds are the central 95% of resampled estimates. The request-time ratio uses summed times alone. Table 7 reports both measurements and output lengths.

Table 7: Additional MATH-500 measurements. A is AR, S is source reuse and R is RelaySpec. Time ratio divides summed AR time by summed relay time.

Drafter	Target	Mean tokens AR / R	Time ratio AR / R	Throughput R / S	Progress S / R
DFlash	8B	783.22 / 778.81	5.018	1.531	7.66 / 6.98
DFlash	14B	777.66 / 772.33	5.143	1.268	7.44 / 6.49
EAGLE-3	8B	783.22 / 767.79	2.397	1.278	5.15 / 4.33
EAGLE-3	14B	777.66 / 766.20	2.625	1.097	5.07 / 4.01

B Complete workload results

Table 8: Native throughput retained and mean recorded progress per cycle. N and R denote native drafting and RelaySpec. Progress includes the implementation's target-supplied token.

Drafter	Target	Native tok/s	Relay tok/s	Retained [95% CI]	Progress N	Progress R
DFlash	8B	206.58	186.49	90.3% [89.8, 90.8]	7.89	6.98
EAGLE-3	8B	94.12	84.92	90.2% [89.8, 90.7]	5.40	4.33
EAGLE-3	14B	66.39	59.18	89.1% [88.6, 89.7]	5.28	4.01

Table 9: Complete DFlash breadth comparisons with paired AR. Ratios use end-to-end throughput. Every planned task and target cell is included.

Target	Task	Requests	AR tok/s	Relay tok/s	Relay / AR [95% CI]	Relay / source
8B	GSM8K	128	37.60	138.89	3.69 [3.59, 3.81]	1.456
8B	HumanEval	164	37.55	120.87	3.22 [3.14, 3.30]	1.244
8B	MBPP	378	37.49	129.35	3.45 [3.36, 3.54]	1.320
8B	MT-Bench	160	37.47	70.69	1.89 [1.76, 2.05]	1.387
14B	GSM8K	128	26.80	96.98	3.62 [3.52, 3.72]	1.104
14B	HumanEval	164	26.73	77.54	2.90 [2.83, 2.98]	0.890
14B	MBPP	378	26.72	83.82	3.14 [3.07, 3.21]	0.956
14B	MT-Bench	160	26.55	49.26	1.85 [1.73, 2.01]	1.086

The earlier fixed-map source comparisons in Table 10 provide all four workloads for both drafter families on RTX 6000 Ada. They are a separate paired experiment with their own denominator. The EAGLE-3 14B code cells show the reduced margin when lower accepted progress outweighs source-removal savings. This operating range complements the AR acceleration measured in the main comparison.

Table 10: Historical paired source-reuse breadth measurements on RTX 6000 Ada. Ratios compare relay with source throughput within these runs. They are not normalized by an AR arm from another experiment.

Drafter	Target	Task	Requests	Source tok/s	Relay tok/s	Ratio [95% interval]
DFlash	8B	GSM8K	128	116.87	166.39	1.4237 [1.4040, 1.4445]
DFlash	8B	HumanEval	164	118.84	145.00	1.2201 [1.2017, 1.2383]
DFlash	8B	MBPP	378	117.77	151.55	1.2868 [1.2732, 1.3001]
DFlash	8B	MT-Bench	160	60.85	81.62	1.3412 [1.3183, 1.3639]
DFlash	14B	GSM8K	128	88.04	97.07	1.1025 [1.0836, 1.1214]
DFlash	14B	HumanEval	164	87.34	77.61	0.8886 [0.8741, 0.9038]
DFlash	14B	MBPP	378	88.02	83.98	0.9541 [0.9416, 0.9663]
DFlash	14B	MT-Bench	160	45.32	49.30	1.0877 [1.0653, 1.1083]
EAGLE-3	8B	GSM8K	128	85.99	108.45	1.2613 [1.2437, 1.2789]
EAGLE-3	8B	HumanEval	164	71.75	81.10	1.1304 [1.1172, 1.1434]
EAGLE-3	8B	MBPP	378	69.85	82.10	1.1753 [1.1656, 1.1850]
EAGLE-3	8B	MT-Bench	160	42.27	49.45	1.1698 [1.1501, 1.1906]
EAGLE-3	14B	GSM8K	128	67.81	73.41	1.0826 [1.0675, 1.0979]
EAGLE-3	14B	HumanEval	164	55.38	50.58	0.9132 [0.9011, 0.9255]
EAGLE-3	14B	MBPP	378	54.55	50.49	0.9256 [0.9151, 0.9361]
EAGLE-3	14B	MT-Bench	160	33.83	34.57	1.0221 [0.9999, 1.0449]

Table 11: Task scores for the historical fixed-map breadth runs. Source reuse and RelaySpec share each reported score. Code columns are single-program pass percentages under base and expanded EvalPlus tests.

Drafter	Target	GSM8K	HumanEval	HumanEval+	MBPP	MBPP+
DFlash	8B	93.75	85.98	80.49	84.13	73.02
DFlash	14B	94.53	89.02	83.54	88.36	75.13
EAGLE-3	8B	92.97	87.20	82.32	83.07	71.96
EAGLE-3	14B	94.53	90.24	85.98	88.89	74.87

Across these historical math/code comparisons, all 4,680 paired task outcomes agree between source reuse and RelaySpec. This count is four model/drafter pairs times $500 + 128 + 164 + 378 = 1,170$ problems. Expanded code tests evaluate the same programs and do not add independent examples. For the current AR-paired DFlash GSM8K runs, the scorer gives 120/128 correct for all arms at 8B and 121/128 at 14B. Code scores above refer specifically to the historical outputs, preserving scorer/run identity.

AR-paired EAGLE-3 code quality. We additionally score the saved programs from the EAGLE-3 AR-paired breadth runs with official EvalPlus base and expanded tests. Table 12 compares one program per task against AR from the same run. At 8B, RelaySpec passes four fewer MBPP tasks under both suites. At 14B, it passes two more. HumanEval expanded-test totals match AR at both target sizes, although individual task outcomes differ. RelaySpec and source reuse have identical per-task outcomes in all eight cells. The paired intervals cover zero. This does not establish equality or noninferiority. These retrospective comparisons use one fitted map per setting and do not include fitting-seed uncertainty. Conversation quality remains unmeasured.

Table 12: EAGLE-3 AR-paired code quality. Counts are passed tasks. The intervals resample paired tasks. Expanded tests reuse the same programs and do not add independent samples.

Target/task	Suite	AR	Relay	Δ (pp)	Paired 95% interval
8B HumanEval	base	143/164	141/164	-1.22	$[-3.66, +1.22]$
8B HumanEval	plus	133/164	133/164	+0.00	$[-3.05, +3.05]$
8B MBPP	base	318/378	314/378	-1.06	$[-2.91, +0.79]$
8B MBPP	plus	276/378	272/378	-1.06	$[-2.65, +0.53]$
14B HumanEval	base	143/164	143/164	+0.00	$[-2.44, +2.44]$
14B HumanEval	plus	134/164	134/164	+0.00	$[-2.44, +2.44]$
14B MBPP	base	338/378	340/378	+0.53	$[-1.32, +2.38]$
14B MBPP	plus	283/378	285/378	+0.53	$[-1.06, +2.12]$

C FITTING AND INTERFACE STUDIES

Table 13: DFlash-8B block-size ablation on 64 questions, a 512-token cap and RTX 6000 Ada hardware. AR is measured in each run.

Block	AR tok/s	Native draft tok/s	Relay tok/s	Relay / AR	Progress / cycle
8	44.68	175.12	164.90	3.691	4.98
16	44.67	229.87	210.15	4.704	6.48
32	44.76	207.70	131.36	2.935	4.14

Distinct records at fixed fitting work. A separate controlled DFlash-8B study holds fitting at 1,024 updates and four records per update while varying distinct records from 64 to 4,096. Smaller sets repeat in the same ordered schedule. Table 14 reports end-to-end throughput on the same 128 development-exposed MATH questions. Of the measured settings, 512 is the smallest within 5% of the best observed mapper throughput. The 256-record setting retains 91.4% of that observed best. This is a descriptive, single-seed boundary, not a minimum across settings or a confirmed optimum. The 512–4,096 settings form an unresolved plateau. Selecting the reference from the same evaluations does not provide independent confirmation. Every relay scores 105/128, versus AR's 104/128.

Table 14: Distinct-data study at 4,096 presentations for every row. Native retention uses each run's paired native-drafter control. Best retention uses the observed best relay across these settings. All results condition on one fitting seed.

Records	Tokens/s	× AR	Native retained	Best retained	Correct
64	134.39	3.55	65.7%	72.8%	105/128
128	154.56	4.10	75.5%	83.7%	105/128
256	168.76	4.51	82.6%	91.4%	105/128
512	181.91	4.84	88.9%	98.5%	105/128
1,024	184.55	4.92	90.3%	99.9%	105/128
2,048	183.81	4.88	89.8%	99.5%	105/128
4,096	184.64	4.90	90.2%	100.0%	105/128

Continuous optimization. We also save checkpoints from one uninterrupted 4,096-record fit at 128, 256, 512, 1,024 and 2,048 updates. They have seen respectively 512, 1,024, 2,048, 4,096 and 4,096 distinct records. The final point therefore measures a second pass, not 8,192 distinct examples. End-to-end AR-relative throughput increases from 3.65 to 5.03 across this trajectory. All five checkpoints score 105/128. Unlike the earlier separately fitted budget configurations, these checkpoints share one optimization trajectory. Figure 7 separates this experiment from the fixed-work data study.

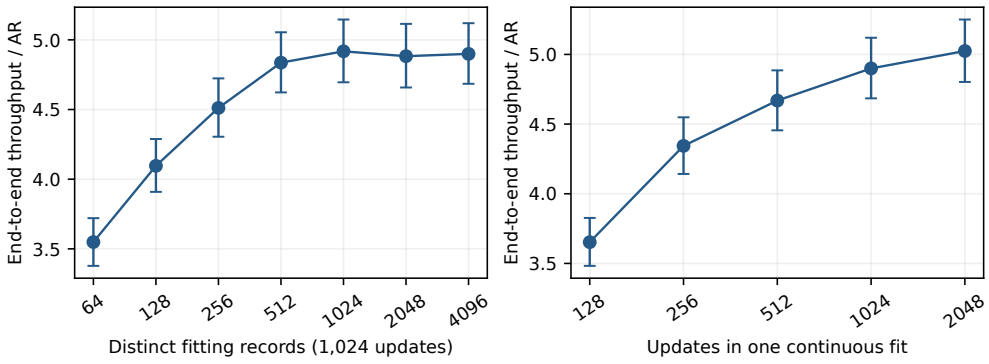

Figure 7: Historical MATH-only fitting study. Left: distinct-data scaling at 1,024 updates. Right: one continuous optimization trajectory. This uses a different training dataset and budget from Figure 4.

Compression should account for the feature distribution. At matched rank 1,024, approximating the mapper's weight matrix alone can be much weaker than preserving its observed projection outputs (Table 15). For the latter, we sample up to eight feature positions from each of 512 fitting records, approximate the dominant uncentered output subspace U_k of $XW^\top$, and deploy $U_k(U_k^\top W)$, using normalized inputs in X where the mapper requires them. This is an application of data-aware compression, with established precedents in ASVD and SVD-LLM (Yuan et al., 2023; Wang et al., 2025b), not a new general factorization method. EAGLE-3 illustrates the distinction: weight-only SVD retains 54.2% on GSM8K, output-subspace compression 93.9%, and direct teacher fitting 96.1%. On MATH the latter two are nearly tied. Direct fitting remains useful alongside posthoc compression.

Drafter / verifier	Task	Weight SVD	Output-subspace	Direct fitting
EAGLE-3 8B	GSM8K	54.1%	93.9%	96.1%
EAGLE-3 8B	MATH	59.4%	96.7%	96.5%
DFlash 14B	MATH	92.7%	96.6%	97.3%

Table 15: Rank-1,024 throughput retained relative to the paired full map on 32 exposed questions with a 128-token cap. Posthoc arms inherit a trained dense map, while direct fitting starts from the original teacher pairs. The deployment rank is matched, not total adaptation compute.

Retargeting toward a smaller verifier. As a boundary experiment, we reuse the Qwen3-4B DFlash drafter with Qwen3-0.6B, taking target layers [1,7,13,19,25]. The source and target tokenizer files are byte-identical at the pinned revisions. We extract 512 fitting and 128 validation records and fit for 8,192 updates. Two dense seeds use 13.11M mapper parameters. Factored linear and GELU MLP maps at width 1,024 each use 7.86M. Per-worker fitting takes 54–61 seconds after cache extraction, which takes 32.8 seconds on four GPUs including model loading.

On 16 development GSM8K requests, dense RelaySpec reaches 168.0 tokens/s, or 3.26 times AR [3.00,3.50], compared with 102.6 tokens/s for source reuse (Table 19). The second seed repeats the result. AR time divided by RelaySpec time is 3.32 [3.07,3.55]. We report both ratios because outputs differ from AR on nine questions in this BF16 run. All methods finish below the 512-token cap. Correctness is 10/16 for speculative methods and 9/16 for AR, with a conservative paired accuracy-difference interval of $[-23.9, 33.8]$ percentage points for dense RelaySpec. This does not establish accuracy superiority or noninferiority.

Source verification occupies 46.8% of source-reuse request time. RelaySpec removes it while reducing progress per cycle from 5.45 to 4.61. This supports the source-removal mechanism even when the verifier is smaller than the drafter's source model. It remains a small, single-workload development result rather than a confirmed deployment recommendation for small verifiers.

To investigate the output disagreement, we repeat the same sixteen questions with the same fitted checkpoint in BF16 and full FP32. All source, draft, target and mapper computations change precision together, and FP32 disables TF32. Within-precision AR agreement is 7/16 in BF16 and 16/16 in

FP32. The BF16 and FP32 AR outputs themselves match on only 5/16 questions. This intervention supports a numerical explanation in this sample, but also changes kernel dispatch and does not isolate target precision alone. It is not a proof of agreement on other prompts. The four diagnostic workers complete in 61–78 seconds. Numerical sensitivity and FP32 mitigation have prior study, including LayerCast (Yuan et al., 2025). Our contribution here is a controlled diagnostic of the inherited-drafter interface, not discovery of this numerical phenomenon or a new precision algorithm.

The pattern extends to the tested code and dialogue screens (Table 16). Code agreement rises from 5/8 to 8/8, and dialogue agreement from 8/16 to 16/16 timed turns across eight conversations. Each precision has its own AR baseline. Code outputs are not execution-scored, and dialogue responses have no quality scores. The results support applicability beyond math in this small verifier, while distinguishing token agreement from task performance.

Workload	Precision	Units	AR matches	Throughput / AR (95% CI)
GSM8K	BF16	16	7/16	3.29 [3.02, 3.54]
GSM8K	FP32	16	16/16	3.37 [3.12, 3.63]
Code	BF16	8	5/8	2.52 [2.25, 2.79]
Code	FP32	8	8/8	2.79 [2.50, 3.10]
Dialogue	BF16	8	8/16	1.94 [1.58, 2.71]
Dialogue	FP32	8	16/16	1.95 [1.57, 2.78]

Table 16: Qwen3-0.6B precision diagnostics with the frozen dense mapper. Units are questions or whole conversations. AR matches count timed turns. Paired intervals resample the units within each precision. GSM8K and code use 512-token caps, dialogue uses 256 tokens per turn.

We also run the same sixteen GSM8K questions with the main Qwen3-8B verifier and its frozen dense mapper. Exact AR agreement rises from 12/16 in BF16 to 16/16 in full FP32. No answer reaches the 512-token cap. The BF16 workers finish in 87–104 seconds and FP32 workers in 152–179 seconds. Runtime records confirm the requested parameter precision and disabled TF32 for FP32. Together with the smaller-verifier screens, this supports the numerical diagnosis across the two tested verifier sizes, without extending the agreement claim beyond these requests and execution settings.

Is output-head precision sufficient? We next promote only the verifier's output projection to FP32, leaving the transformer, input embeddings and mapper in BF16. We copy the output head before promotion to preserve input-embedding precision if the weights are tied. On the same sixteen GSM8K questions, agreement with each runtime's own AR rises from 7/16 to 9/16 for Qwen3-0.6B and from 12/16 to 16/16 for Qwen3-8B (Table 17). Head-only FP32 does not reproduce full-FP32 AR: those matches are 7/16 and 13/16, respectively. Thus the output projection is a useful intervention in the tested 8B setting, but does not explain all precision sensitivity or provide full-FP32 equivalence. These sixteen requests were already exposed during diagnosis.

Target	Head	Own AR matches	FP32 AR matches	Tokens/s	Throughput / AR
0.6B	BF16	7/16	5/16	168.1	3.24
0.6B	FP32	9/16	7/16	161.5	3.23
8B	BF16	12/16	9/16	164.6	4.30
8B	FP32	16/16	13/16	154.8	4.28

Table 17: Output-head precision with a BF16 transformer. Own AR uses the same runtime as RelaySpec. FP32 AR runs the entire model in FP32 and is a distinct reference. Rates concern the same questions but can time different outputs across precision settings.

The initial 8B agreement does not generalize to a broader development screen (Table 18). On 32 additional GSM8K requests, FP32-head agreement with own-runtime AR rises from 15/32 to 20/32; on sixteen MATH requests it rises from 3/16 to 5/16. Released native DFlash and RelaySpec have identical output strings on every request within each precision setting, including their disagreements with AR. This points to a shared execution issue in these samples, rather than an effect unique to retargeting. Head promotion is only a partial mitigation. Native DFlash also uses the target output

head for proposal logits, so its intervention changes that projection too. The MATH screen is heavily truncated at 512 tokens and does not measure full-answer accuracy.

Workload	Head	Units	Native matches	Relay matches	AR capped
GSM8K	BF16	32	15/32	15/32	1/32
GSM8K	FP32	32	20/32	20/32	1/32
MATH	BF16	16	3/16	3/16	11/16
MATH	FP32	16	5/16	5/16	11/16

Table 18: Broader 8B head-precision diagnostic. Matches compare complete recorded token sequences with each runtime's own AR, including capped sequences. MATH AR reaches the cap on 11/16 requests in both settings; native and RelaySpec reach it on 11/16 in BF16 and 10/16 with the FP32 head. One GSM8K request is capped for every method in both settings.

Method	Tokens/s	Throughput / AR	AR time / time	Correct	AR matches
AR	51.6	1.00	1.00	9/16	16/16
Source reuse	102.6	1.99	2.03	10/16	7/16
Dense, seed 1729	168.0	3.26	3.32	10/16	7/16
Dense, duplicate	167.0	3.24	3.30	10/16	7/16
Dense, seed 1730	168.2	3.26	3.32	10/16	7/16
Factored linear-1024	161.2	3.13	3.18	10/16	7/16
MLP-1024	147.9	2.87	2.92	10/16	7/16

Table 19: Downward retargeting to Qwen3-0.6B on 16 GSM8K requests. Timing includes prompt processing. Correctness and exact AR-output matches are separate. The duplicate row repeats the same checkpoint and passes the trajectory-equivalence check. All models run in BF16.

Selecting target features. The interface also permits a controlled test of target-layer sufficiency. We refit DFlash's linear mapper using only target layers 25 and 33, keeping the released drafter frozen, the same 512 fitting records and 8,192 batch-four updates. This reduces the mapper from 52.43M to 20.97M parameters. It does not remove target-model layers or reduce whole-model parameters by 60%. Initial development screens selected this two-layer interface before confirmation.

A separate protocol then fixed both checkpoint hashes, the first 64 lexicographically ordered IDs from the previously unused GSM8K reserve, a 2,048-token cap, and a 95% throughput-retention criterion. An updated exposure audit found no reserve IDs in 964 collected rank files across both worktrees. The original fitting-data exact/lexical exclusions remain inherited. Four disjoint 16-question shards each compare the same two methods on one L40S. The paired bootstrap resamples all 64 requests 10,000 times with seed 1729. The lower 95% endpoint exceeds the fixed 95% retention threshold (Table 20). Both maps produce identical token sequences on all 64 questions and score 62/64. None reaches the cap. A conservative paired accuracy-difference interval is $[-6.62, 6.62]$ percentage points, so this sample does not establish one-point quality noninferiority.

Interface	Parameters	Tokens/s	Retention (95% CI)	Correct
Five target layers	52.43M	160.67	100.00 [100.00, 100.00]%	62/64
Two target layers	20.97M	156.86	97.63 [96.23, 99.03]%	62/64

Table 20: Fixed DFlash-8B interface comparison on 64 fresh GSM8K requests. Parameter counts concern the mapper. Throughput includes request overhead. Intervals resample requests and do not include fitting-seed variation.

The recorded profiles explain why reducing mapper size need not improve end-to-end speed. In this fixed comparison, full-map computation occupies 0.74% of total request time, including its prefill application, while full-target verification occupies 81.9%. The two-layer map reduces its own share to 0.58%, but modestly lower progress per verification cycle raises total verification time. Preserving

acceptance therefore matters more than mapper arithmetic in this setting. These are recorded region attributions, not isolated kernel or memory measurements.

The result has a family-dependent boundary. On 16 exposed, 128-token MATH requests, two-layer refitting retains 99.3% of full-map throughput with a 14B DFlash verifier, versus 93.7% with EAGLE-3. A second 8B fitting seed retains 97.6% for DFlash and 94.8% for EAGLE-3 on 16 exposed GSM8K requests. These are development screens, not additional confirmation tests. They motivate studying which features a frozen interface requires rather than asserting that two target layers always suffice.

At equal two-layer mapper size, depth matters (Figure 8). Refitting from early layers 1 and 9 retains 56.3% of full-map throughput for DFlash and 57.0% for EAGLE-3. Spaced layers 9 and 25 retain 91.5% and 91.4%, while late layers 25 and 33 retain 98.7% and 96.2% on the same 16 development questions. All fits use 512 records, 8,192 updates and seed 1729. This controls mapper capacity, supporting an information-depth explanation within these settings. It does not establish that this particular layer pair is optimal across tasks or models.

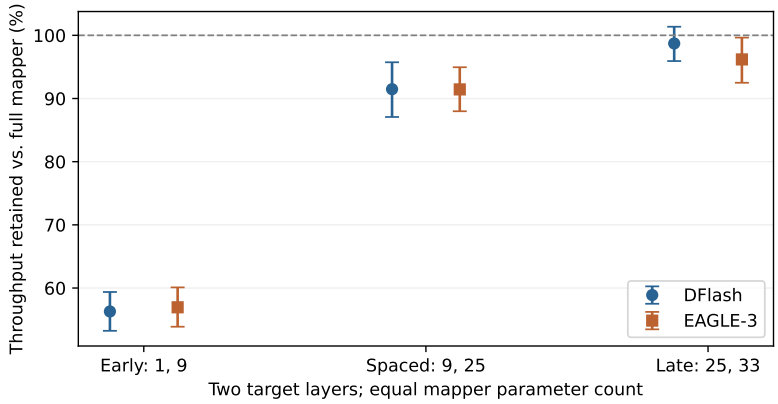

Figure 8: Equal-capacity target-layer comparison on 16 exposed GSM8K questions with a 128-token cap. Each arm is paired with its own full-map baseline. Late-arm rows are the matching subset of the earlier 32-request screen. Paired 95% request intervals exclude fitting-seed variation.

Can nonlinearity compensate for early-layer inputs? We fit an early-layer GELU MLP and a factorized linear map at width 1,950, giving exactly 20,966,400 parameters in each case. Both use layers [1,9], 512 records and 8,192 updates at seed 1729. Each decoding worker also evaluates the existing early- and late-layer dense maps. The MLP lowers training error in both drafter families, but does not close the gap to late-layer inputs (Table 21). It worsens DFlash validation error. For EAGLE-3 it slightly improves validation error, yet throughput remains far below the late-layer control. Late-layer dense maps retain 97.4–98.6% in the DFlash workers and 92.6–92.7% in EAGLE-3, compared with 50.0–57.9% for the fitted early-layer maps. This limits the tested nonlinear fitting approach, not the recoverability of later-layer information in principle.

Family	Early-layer map	Train error	Val. error	Progress/cycle	Retention
DFlash	Factored linear	0.333	0.409	2.61	56.0%
DFlash	GELU MLP	0.271	0.439	2.33	50.0%
EAGLE-3	Factored linear	0.333	0.411	2.57	57.9%
EAGLE-3	GELU MLP	0.272	0.399	2.56	57.4%

Table 21: Early-layer reconstruction at matched parameter count. Retention is relative to each worker's paired full map on eight exposed GSM8K requests with a 128-token cap. Progress includes the committed target token. Errors use the same fitting and validation diagnostics within each family. One fitting seed is tested.

Cross-domain limits of layer sufficiency. Without changing the Numina-fitted maps, we repeat the depth screen on code and dialogue (Table 22). DFlash's late-layer map retains 99.7% on code and 97.8% on dialogue. EAGLE-3 retains 85.4% and 91.9%, and its spaced-layer map is slightly faster on dialogue. Thus late-layer sufficiency transfers well in the tested DFlash screens, but is not a family-independent rule. These small exposed screens do not establish code test-pass rates, dialogue quality, or a generally optimal layer choice.

Drafter	Workload	Early	Spaced	Late
DFlash	Code	40.5%	84.8%	99.7%
DFlash	Dialogue	65.8%	91.3%	97.8%
EAGLE-3	Code	52.0%	84.9%	85.4%
EAGLE-3	Dialogue	69.6%	93.0%	91.9%

Table 22: Throughput retained relative to each arm's paired full mapper. Code uses eight MBPP requests with a 512-token cap. Dialogue uses eight two-turn MT-Bench conversations with a 256-token per-turn cap. Early, spaced and late denote layers $[1, 9]$, $[9, 25]$ and $[25, 33]$. All compact maps have equal parameter count and the same fitting data and budget.

Native interface compression as an application. The same interface abstraction supports a secondary experiment without retargeting: compress the released Qwen3-8B DFlash input projection by rank-1,536 SVD, leaving the rest of the native drafter unchanged. The rank was fixed from development screens before generating any of the 64 confirmation answers above. We retain both the released native path and a full-rank projection repacked into the relay path as controls. The compressed projection has 37.75M rather than 83.89M parameters (55% fewer). This is not a whole-drafter parameter reduction.

Table 23 uses the same 64 requests and output cap. Relative to the released native path, compression retains 98.66% [97.58, 99.75]% end-to-end throughput. All three methods score 62/64 with identical outputs and no cap-length outputs. Repacking alone retains 99.16%, so the table distinguishes packaging overhead from compression. This is a preselected secondary comparison, not another primary significance test. It demonstrates an application of inspecting and replacing the feature interface. SVD itself is not a novel method, and activation-aware LLM compression already has established precedents (Yuan et al., 2023; Wang et al., 2025b).

Interface	Parameters	Tokens/s	Retention (95% CI)	Correct
Released native	83.89M	178.65	100.00 [100.00, 100.00]%	62/64
Repacked native	83.89M	177.16	99.16 [99.00, 99.28]%	62/64
Rank 1,536	37.75M	176.25	98.66 [97.58, 99.75]%	62/64

Table 23: Preselected native DFlash projection compression on 64 GSM8K requests. Retention is relative to the released native implementation. Only projection parameters are counted.

Distilling a native interface from fewer layers. The layer result also suggests an alternative to weight decomposition: fit a smaller interface directly to the released native projection's outputs. Using cached target features from 512 fitting records, we apply the frozen native projection and its output normalization to construct supervision, then fit a one-layer or two-layer input map for 8,192 updates. The native drafter remains frozen. Teacher hashes, target identities and layer lists are checked, and the fitting path passes its batched versus individual-example gradient check.

The two-layer native map has 33.55M parameters (60% fewer than the 83.89M native projection), while one layer has 16.78M (80% fewer). Table 24 shows the tradeoff on development workloads: two layers retain 97.1–97.9% throughput relative to released native, while one layer loses substantially more, especially on code. Rank-1,536 SVD retains slightly more throughput with slightly more parameters (37.75M). Thus the student provides another compact-interface operating point, not a uniform improvement over decomposition. These students were selected after the fixed native-SVD comparison and are not evaluated as new candidates on its confirmation questions.

Workload	Units	One layer	Two layers	SVD rank 1,536
GSM8K	32	84.7%	97.7%	98.8%
MATH	32	88.8%	97.1%	98.1%
Code	8	71.6%	97.2%	98.6%
Dialogue	8	87.0%	97.9%	99.2%

Table 24: Native interface students versus released-native throughput. GSM8K/MATH use 32 questions and a 128-token cap, code eight MBPP requests and a 512-token cap, and dialogue eight two-turn conversations with a 256-token per-turn cap. All are exposed development workloads.

Frozen native-student confirmation. We subsequently freeze the two-layer student's checkpoint and a new 64-question set before decoding. These are sorted reserve positions 64–127, disjoint from the preceding interface/SVD confirmation. A scan of 932 prior local result files finds no matching question IDs; this does not establish global or pretraining non-exposure. The primary criterion requires the lower endpoint of the paired 95% throughput interval against released native to exceed 95%. We retain the one-layer student, rank-1,536 SVD and full repacked projection as descriptive controls, with no optional sample extension or candidate refitting.

The two-layer student retains 97.05% [95.93,98.21]% of released-native throughput and passes the fixed criterion (Table 25). One layer retains only 84.55% [83.11,86.10]%, while SVD retains 98.08% [97.13,99.06]% with slightly more parameters than two layers. Every method produces identical recorded outputs, scoring 60/64, and none reaches the 2,048-token cap. The conservative paired accuracy interval for two layers versus released native is still $[-6.62, +6.62]$ percentage points: identical observed answers do not establish tight accuracy noninferiority. Together with retargeting, this supports a compact late-layer conditioning interface in two uses, without claiming that two layers suffice for every drafter or verifier.

Interface	Parameters	Tokens/s	Retention (95% CI)	Correct
Released native	83.89M	170.22	100.00 [100.00, 100.00]%	60/64
Repacked native	83.89M	168.91	99.23 [99.18, 99.28]%	60/64
Two-layer student	33.55M	165.21	97.05 [95.93, 98.21]%	60/64
One-layer student	16.78M	143.93	84.55 [83.11, 86.10]%	60/64
Rank 1,536	37.75M	166.96	98.08 [97.13, 99.06]%	60/64

Table 25: Fixed native-student comparison on a separate set of 64 GSM8K questions. Retention uses the released-native reference and 10,000 paired question bootstrap replicates, seed 1729. Only the two-layer comparison is primary. Counts refer to the input projection, not the whole drafter; fitting-seed variation is outside these intervals.

A smaller-verifier boundary. Repeating the depth screen with the 0.6B verifier gives a weaker compact interface (Table 26). At matched 5.24M parameters, early, spaced and late two-layer maps retain 52.5%, 79.9% and 82.2% of the full five-layer mapper's throughput. One layer retains 60.7% with 2.62M parameters. Late features remain more useful in this screen, but the near-full retention observed at 8B does not carry over to this smaller verifier. This is an eight-question development result with one fitting seed, not a general lower bound on the required layers.

Verifier	Target layers	Train error	Val. error	Progress/cycle	Retention
Qwen3-0.6B	Early [1,7]	0.432	0.459	2.34	52.5%
Qwen3-0.6B	Spaced [7,19]	0.355	0.374	3.70	79.9%
Qwen3-0.6B	Late [19,25]	0.332	0.351	3.76	82.2%
Qwen3-0.6B	Last [25]	0.417	0.430	2.75	60.7%

Table 26: Downward-retargeting depth screen, 4B-trained DFlash to a 0.6B verifier. All fits use 512 records and 8,192 updates; decoding uses eight exposed GSM8K requests with a 512-token cap. Retention is relative to each worker's paired full mapper. Errors use the common fitting and validation sets.

A matched-capacity nonlinear follow-up does not recover that deficit (Table 27). Width-1,137 MLP and factorized maps each have 5,239,296 parameters, versus 5,242,880 for late dense. MLPs lower training error from about 0.332 to 0.303–0.304 and validation error from 0.351 to 0.345, yet retain only 81.1–83.1% throughput at two fitting seeds. Factorized maps retain 83.1–84.2%; their paired late-dense controls retain 82.8–84.3%. The same eight questions are reused across seeds. This closes the tested fitting intervention, without establishing that later features are unrecoverable in principle.

Seed	Architecture	Train error	Val. error	Progress/cycle	Retention
1729	GELU MLP	0.304	0.345	3.69	81.1%
1729	Factorized linear	0.336	0.353	3.76	84.2%
1730	GELU MLP	0.303	0.345	3.78	83.1%
1730	Factorized linear	0.336	0.353	3.77	83.1%

Table 27: Matched-size nonlinear follow-up for 0.6B late-layer mapping. Both architectures use layers [19,25], 512 records and 8,192 updates. Retention is relative to each worker's full mapper on the same eight exposed questions; fitting seeds do not add independent evaluation samples. Better feature regression does not close the decoding deficit.

Expanded NuminaMath data curve. Table 28 reports all eleven completed sizes at 8,192 batch-four updates, spanning 2,048 epochs at 16 records to one epoch at 32,768. The same 1,024 feature-validation records are used throughout. Larger data marginally improves feature loss but does not improve measured decoding beyond the plateau. The 512-record mapper retains 97.2% of the observed best throughput, while the 256-record mapper retains 93.9%. These percentages are descriptive and conditional on the evaluated fitting seed and development questions.

Table 28: NuminaMath fitting at fixed 8,192 updates on 128 paired MATH questions with a 2,048-token output cap. All eleven mapper outputs score 105/128, versus AR's 104/128.

Records	Tokens/s	Speedup / AR	Correct / 128
16	96.87	2.58×	105
32	133.23	3.55×	105
64	157.74	4.20×	105
128	177.75	4.73×	105
256	186.36	4.96×	105
512	192.81	5.14×	105
2,048	196.93	5.25×	105
4,096	198.30	5.28×	105
8,192	197.98	5.27×	105
16,384	198.37	5.28×	105
32,768	197.94	5.27×	105

Table 29: Completed DFlash-8B fitting studies on the same 128-question set. Records count distinct fitting examples. Two passes over 4,096 records give 8,192 presentations. Source reuse is paired within each run.

Fitting choice	Records	Updates	Relay tok/s	Relay / source [95% CI]	Score
512 records	512	128	136.78	1.150 [1.118, 1.181]	82.03
1,024 records	1,024	256	162.04	1.372 [1.347, 1.397]	82.03
2,048 records	2,048	512	173.72	1.477 [1.458, 1.495]	82.03
4,096 records, two passes	4,096	2,048	187.85	1.573 [1.558, 1.588]	82.03
Ridge surrogate	4,096	*solve*	115.18	0.955 [0.927, 0.982]	82.03
Nonlinear width 512	4,096	1,024	104.49	0.872 [0.841, 0.903]	82.03

The data-budget configurations use the same normalized linear interface and relative-error objective. The nonlinear map uses an intermediate width of 512 and about 11.8M parameters, compared with 52.4M for the dense 8B map. Its lower throughput is consistent with a less effective fitted interface,

but the comparison also changes capacity. It does not isolate a disadvantage of nonlinear functions. The ridge configuration solves a raw linear surrogate with ridge coefficient 1.0. Since the deployed DFlash interface includes output normalization, this is a different objective from Equation 3.

Matched capacity on a separate fitting source. We additionally fit 30 DFlash-8B configurations on nested sets of 512 and 2,048 NuminaMath-CoT records,[1] using a fixed 1,024-record fitting validation split. The source-stratified sets include `cn_k12`, `orca_math` and `synthetic_math` examples. Their lexical filter excludes exact normalized problem/solution matches and near matches at five-shingle Jaccard similarity at least 0.6 for texts with at least ten unique shingles. This filter does not establish semantic or template independence. These results use a separate fitting source from the historical MATH study above.

Each configuration receives 8,192 updates with four records per update: 64 passes at $N = 512$ and 16 at $N = 2048$. Frozen features are cached once. Factored linear maps and bias-free two-layer GELU MLPs use widths 64, 128, 256, 512, 1,024, 2,048 and 4,096, giving matched parameter counts $h(20{,}480 + 2{,}560)$. The dense reference has 52.43M parameters. The rank of a linear function remains bounded by 2,560 even when its factorization width is larger. All cells share the data order, relative-interface objective, learning rate 6×10^{-4} and seed 1729.

Table 30: Validation relative interface MSE for the capacity study. Linear and MLP entries at the same width have equal parameter counts. The dense row supplies the single-linear-layer reference.

Width	Params (M)	Linear, N=512	MLP, N=512	Linear, N=2048	MLP, N=2048
64	1.47	0.5732	0.5790	0.5692	0.5752
128	2.95	0.4761	0.4871	0.4690	0.4804
256	5.90	0.3760	0.3883	0.3648	0.3767
512	11.80	0.2862	0.3082	0.2685	0.2878
1024	23.59	0.2295	0.2579	0.2040	0.2224
2048	47.19	0.2133	0.2438	0.1847	0.1982
4096	94.37	0.2145	0.2501	0.1845	0.1936
Dense	52.43	0.2101	—	0.1805	—

At this optimization budget, wider factorizations approach the dense reference, while MLPs have higher validation error at every matched width. Increasing MLP width from 2,048 to 4,096 at $N = 512$ lowers diagnostic training error from 0.1586 to 0.1278 but raises validation error from 0.2438 to 0.2501. At $N = 2048$, validation error instead decreases from 0.1982 to 0.1936. This interaction motivates checking regularization and convergence before attributing the ranking to function class.

More epochs on fixed smaller datasets. We continue the width-512 linear and MLP fits to 32,768 total updates, preserving Adam moments, random states and cyclic data position. A separate bounded pilot reproduced uninterrupted fitting bit for bit. Figure 9 shows late-training diagnostics on the same first 256 training records and all 1,024 validation records. The $N = 2048$ MLP improves from 0.2878 to 0.2787 validation error between 8,192 and 32,768 updates. At $N = 512$, linear training error falls from 0.2443 to 0.2350 while validation error rises from 0.2862 to 0.2873. The latter is consistent with mild overfitting on this diagnostic split. These single-seed fitting trajectories require separate decoding and quality evaluation, including replication across targets and workloads.

[1] `AI-MO/NuminaMath-CoT`, revision `9d8d210c9f6a`, training shard 0 of 5.

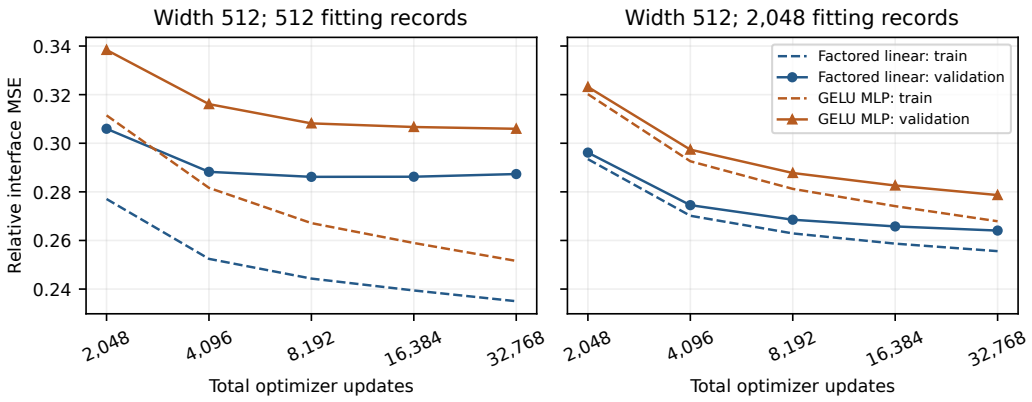

Figure 9: Late-training trajectories, starting at 2,048 updates. Dashed lines use the fixed training diagnostic subset. Solid lines use fitting validation. The horizontal axis counts updates, while the distinct datasets remain fixed at 512 or 2,048 records.

Explicit regularization on fixed smaller datasets. We complete 42 nonzero-penalty fits for the dense, width-512 factored linear and width-512 GELU MLP connectors at $N \in \{512, 2048\}$. Each fit uses 8,192 updates, the same 1,024 validation records and seed 1729. Explicit L2 coefficients are $10^{-7}, 10^{-6}, 10^{-5}, 10^{-4}$. Separate AdamW arms use decay $10^{-4}, 10^{-3}, 10^{-2}$. No tested L2 coefficient improves its matched zero-penalty endpoint. The largest AdamW improvement is a 0.17% relative decrease in validation error for the width-512 linear connector at $N = 512$. Neither MLP setting improves with either penalty family. Table 31 reports the best validation value within each penalty family, so these minima are development selections. Figure 10 retains all tested coefficients. This grid does not establish a decoding gain or a penalty effect across seeds, and it does not resolve regularization for the wider MLPs.

Table 31: Matched regularization endpoints. Lower relative interface MSE is better. Each nonzero minimum is selected from four L2 or three AdamW coefficients using the same fitting validation split.

N	Mapper	No penalty	Best tested L2	Best tested AdamW
512	Dense	0.21008	0.21138	0.21011
512	Factored linear 512	0.28617	0.28738	0.28570
512	GELU MLP 512	0.30818	0.30827	0.30822
2048	Dense	0.18055	0.18424	0.18055
2048	Factored linear 512	0.26853	0.26970	0.26852
2048	GELU MLP 512	0.28776	0.28864	0.28779

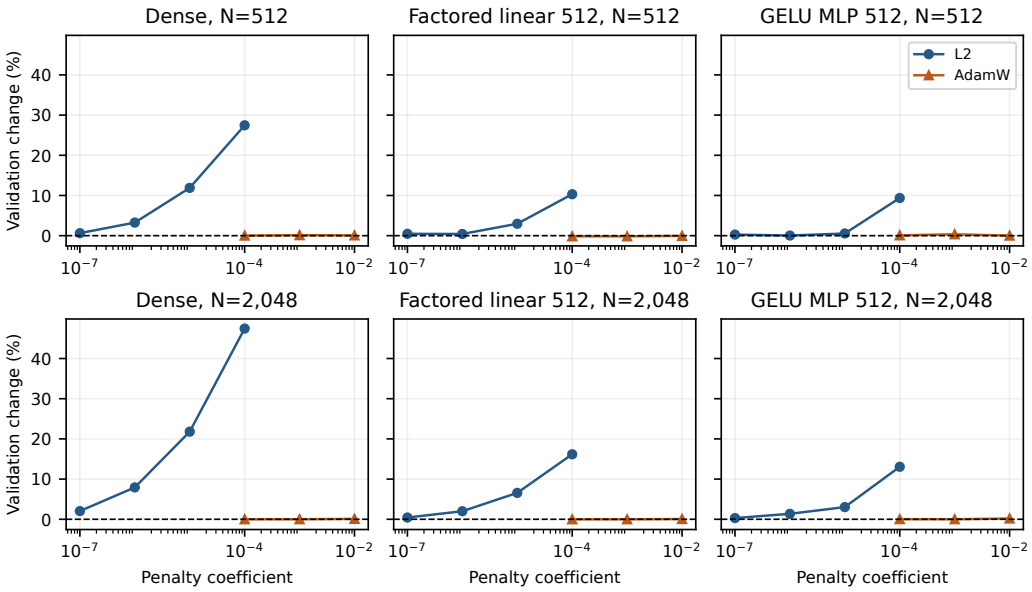

Figure 10: All 42 penalty endpoints relative to their matched zero-penalty validation error. Positive values indicate worse fitting validation. The two curves are separate objective-L2 and AdamW experiments.

Table 32: Matched EAGLE-3 8B normalization comparison on 32 development questions after 128 updates. Throughput ratios use source reuse.

Input to linear map	Relative error	Throughput ratio [95% interval]	Progress retained
Normalized features	0.413	1.035 [1.003, 1.068]	69.0%
Raw features	0.266	1.237 [1.213, 1.263]	82.6%

Table 33: DFlash objective comparison on 32 development questions. Candidates share fitting data, update count, normalization and optimizer. Each ratio uses the source-reuse arm of that candidate's paired run.

Target	Objective	Throughput ratio [95% interval]	Token matches
8B	Relative error	1.523 [1.488, 1.558]	32/32
8B	Squared error + cosine	1.509 [1.476, 1.545]	32/32
14B	Relative error	1.265 [1.243, 1.289]	32/32
14B	Squared error + cosine	1.251 [1.229, 1.276]	32/32

Relative error weights a record's feature mismatch by teacher magnitude. Squared error plus 0.1 times cosine distance combines coordinate error with directional agreement. Direct candidate-to-candidate throughput ratios are 1.0085 [1.0005, 1.0174] at 8B and 1.0114 [1.0046, 1.0177] at 14B. These are modest gains with one fewer mixing coefficient.

C.1 EAGLE-3 FITTING REPLICATION

We repeat the fixed smaller-data fitting study with the EAGLE-3 source drafter and the 8B target. The grid contains dense, factored linear and GELU MLP connectors, with widths 128, 512 and 2048 for the latter two. Each primary fit uses 8,192 updates at $N \in \{512, 2048\}$ and the same 1,024 fitting-validation records. Two additional seeds repeat the dense $N = 2048$ setting. This experiment uses the EAGLE input convention without the DFlash input normalization. Error values are interpreted within each drafter family's conditioning interface.

Table 34: EAGLE-3 fitting validation. End denotes the 8,192-update endpoint. Best denotes the minimum saved fitting-validation error, with its update count. These are development selections for seed 1729, not decoding results. M weights counts millions of trainable mapper parameters.

		$N = 512$			$N = 2048$		
Mapper	M weights	End	Best	Update	End	Best	Update
Dense	52.43	0.24787	0.21666	1024	0.21203	0.21061	2048
Linear 128	2.95	0.41980	0.41980	8192	0.41505	0.41505	8192
Linear 512	11.80	0.28632	0.28505	2048	0.27188	0.27188	8192
Linear 2048	47.19	0.23486	0.22877	2048	0.21132	0.21132	8192
MLP 128	2.95	0.46554	0.46554	8192	0.45645	0.45645	8192
MLP 512	11.80	0.31749	0.31749	8192	0.29577	0.29577	8192
MLP 2048	47.19	0.28154	0.27124	2048	0.23196	0.22683	4096

The dense $N = 512$ mapper reaches 0.21666 at 1,024 updates, then rises to 0.24787 at the final update. Its endpoint is worse than the width-2048 linear mapper, but its best saved validation value is better than that linear mapper's 0.22877. At $N = 2048$, the dense and width-2048 linear minima are 0.21061 and 0.21132. Thus endpoint rankings alone can reflect different convergence trajectories. At every matched width in this grid, the MLP's best saved validation error exceeds the linear mapper's minimum. The three dense $N = 2048$ endpoints range from 0.212034 to 0.212043. This narrow seed spread concerns one fitting setting. It does not establish seed stability for the other architectures or for decoding performance.

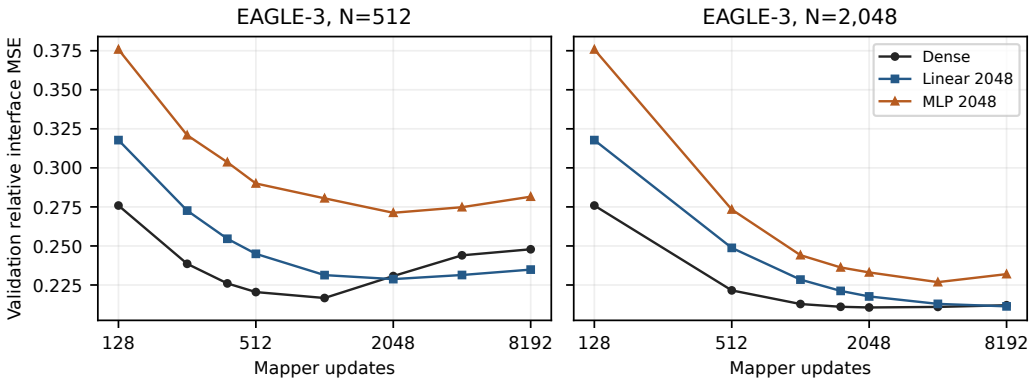

Figure 11: EAGLE-3 fitting-validation trajectories at fixed distinct-data counts. Additional updates can worsen validation, most visibly for the dense mapper at $N = 512$. The horizontal axis counts updates rather than new examples. The subsequent checkpoint comparison measures actual decoding.

C.2 Feature-validation minima and decoding

We compare each distinct saved feature-validation minimum with its 8,192-update endpoint in two declared decoding campaigns. Each uses 16 paired development questions, greedy decoding and a 256-token cap. Checkpoint selection precedes decoding. Table 35 includes every distinct early-versus-endpoint pair in these campaigns.

Table 35: Decoding throughput at the minimum feature-validation checkpoint relative to the same fit's endpoint. Intervals resample paired requests 10,000 times. They are descriptive, without multiple-comparison adjustment, and do not measure variation across fitting seeds.

Family	Mapper	N	Learning rate	Selected update	Early / end [95% CI]
DFlash	Linear 4096	512	0.0006	4096	1.009 [0.981, 1.040]
DFlash	MLP 4096	512	0.0006	4096	0.978 [0.961, 0.998]
DFlash	Linear 4096	512	0.0002	4096	1.008 [0.985, 1.029]
DFlash	Linear 4096	512	0.0018	4096	1.003 [0.978, 1.029]
DFlash	MLP 4096	512	0.0002	2048	1.011 [0.986, 1.039]
DFlash	MLP 4096	512	0.0018	4096	0.961 [0.939, 0.983]
EAGLE-3	Dense	512	0.0006	1024	0.950 [0.924, 0.976]
EAGLE-3	Linear 512	512	0.0006	2048	0.969 [0.948, 0.988]
EAGLE-3	Linear 2048	512	0.0006	2048	0.964 [0.922, 1.003]
EAGLE-3	MLP 2048	512	0.0006	2048	0.968 [0.942, 0.996]
EAGLE-3	Dense	2048	0.0006	2048	0.980 [0.969, 0.992]
EAGLE-3	MLP 2048	2048	0.0006	4096	1.019 [0.995, 1.046]

Lower feature-validation loss does not consistently predict faster decoding. For EAGLE-3's dense $N = 512$ fit, the selected 1,024-update checkpoint retains 94.99% of endpoint throughput, with interval [92.39, 97.59]%. For the DFlash width-4096 MLP at $N = 512$ and learning rate 0.0006, the selected 4,096-update checkpoint retains 97.77%, with interval [96.09, 99.76]%. Other intervals cross equal throughput. These results support treating feature regression as a surrogate and measuring decoding when selecting a connector. They do not establish full-answer quality or justify selection on untouched evaluation data.

Table 36: Observed throughput ranges across three fitting seeds in a separate 16-question, 256-token development campaign. Selected uses each fit's minimum saved feature-validation checkpoint. These ranges describe the three measured fits and runtime variation, not confidence intervals.

Mapper	N	Learning rate	Endpoint tok/s range	Selected tok/s range
Dense	512	0.0006	184.34–184.68	183.36–184.58
Dense	2048	0.0006	186.11–189.22	186.11–189.22
Linear 1024	512	0.0006	179.47–181.23	179.47–181.23
Linear 4096	2048	0.0002	185.36–187.81	185.36–187.81
MLP 4096	2048	0.0002	183.92–184.78	183.92–184.78
MLP 4096	512	0.0002	174.53–177.30	175.57–177.56

All six settings in Table 36 use the same cached features and seeds 1729, 1730 and 1731. The dense $N = 512$ endpoints span 184.34–184.68 tokens/s. The width-4096 $N = 2048$ MLP improves on its $N = 512$ counterpart, while the dense connector remains competitive. These short development runs do not establish the minimum sufficient data count or distinguish small architecture gaps from runtime variation.

C.3 Longer development outputs for small-data mappers

We evaluate nine saved DFlash connectors on 128 paired MATH development questions with a 2,048-token cap, greedy decoding and the same source, target and native-drafter references. These exposed questions provide a larger development comparison than the 16-question, 256-token screen. They are separate from any untouched confirmation set. The continued width-512 fits reuse the same 2,048 distinct training examples.

Table 37: Capped development outcomes for small-data DFlash fitting. Throughput retention uses the dense $N = 2048$, 8,192-update connector. Intervals use 10,000 paired request bootstrap samples. Correct counts scored answers. Cap counts outputs reaching 2,048 tokens, which can include EOS exactly at the limit and are not exact truncation counts.

Method	N	Updates	Tok/s	Dense retained (%) [95% CI]	Correct	Cap
AR	–	–	37.72	19.04 [18.23, 19.90]	104/128	11
Native drafter	–	–	204.57	103.27 [102.35, 104.17]	105/128	9
Source reuse	–	–	120.96	61.06 [60.53, 61.59]	105/128	9
Dense	512	8192	193.68	97.78 [97.09, 98.47]	105/128	9
Dense	2048	8192	198.09	100.00 [100.00, 100.00]	105/128	9
Linear 1024	512	8192	188.19	95.00 [94.14, 95.85]	105/128	9
Linear 4096	2048	8192	196.25	99.07 [98.50, 99.60]	105/128	9
MLP 4096	2048	8192	190.15	95.99 [95.30, 96.70]	105/128	9
Linear 512	2048	8192	164.61	83.10 [81.81, 84.38]	105/128	9
Linear 512	2048	32768	167.29	84.46 [83.15, 85.71]	105/128	9
MLP 512	2048	8192	154.39	77.94 [76.50, 79.35]	105/128	9
MLP 512	2048	32768	157.68	79.60 [78.07, 81.06]	105/128	9

Dense fitting on 512 examples retains 97.78% of the dense $N = 2048$ throughput, with interval [97.09, 98.47]%. This establishes a lower-data performance point in this setting, but does not establish that 512 is minimal. The width-1024 linear mapper at $N = 512$ retains 95.00%, with interval [94.14, 95.85]%, leaving its 95% boundary unresolved. The width-4096 linear mapper is slightly below the dense connector in this larger sample, so the earlier short-screen point ranking should not be interpreted as an established architecture advantage.

All nine connectors score 105 of 128 answers, compared with 104 for AR. Their paired accuracy-difference intervals are $[-0.03125, 0.046875]$. These intervals do not establish a one-percentage-point noninferiority margin. Nine connector outputs and eleven AR outputs reach the token cap. The observed scores therefore concern the configured capped evaluation, not uncapped answer quality. Longer width-512 fitting gives modest throughput increases while remaining below the wider or dense connectors. This comparison does not treat additional updates as additional data.

C.4 EAGLE-3 LONGER-OUTPUT CAPACITY COMPARISON

Two disjoint 64-question shards evaluate eight saved EAGLE-3 maps and three shared controls on the same 128 exposed MATH development questions. All 1,408 expected method/request pairs are present. The 2,048-token cap, fitting checkpoints, source revisions and pinned scorer remain fixed across shards. An eight-request pilot precedes this evaluation.

Table 38: EAGLE-3 with an 8B target on 128 exposed development questions. Dense retention uses the $N = 2048$ endpoint. Intervals are paired 95% request-bootstrap intervals conditional on the fits. Cap counts all outputs reaching 2,048 tokens. It is not an exact truncation count.

Method	N	Tokens/s	Dense retained (%) [95% CI]	Correct	Cap
AR	–	36.32	38.19 [37.14, 39.25]	104/128	11
Native drafter	–	94.89	99.78 [98.86, 100.80]	105/128	7
Source reuse	–	66.80	70.24 [69.69, 70.84]	105/128	7
Dense	512	93.47	98.28 [97.59, 99.01]	105/128	7
Dense	2048	95.10	100.00 [100.00, 100.00]	105/128	7
Linear 2048	512	93.85	98.69 [98.08, 99.31]	105/128	7
Linear 2048	2048	93.59	98.41 [97.76, 99.12]	105/128	7
MLP 2048	512	84.67	89.03 [88.24, 89.87]	105/128	7
MLP 2048	2048	89.25	93.85 [93.27, 94.46]	105/128	7
Linear 512	2048	84.91	89.29 [88.54, 90.09]	105/128	7
MLP 512	2048	79.76	83.87 [82.94, 84.84]	105/128	7

Dense $N = 512$ retains 98.28% of dense $N = 2048$ throughput. Linear-2048 at $N = 512$ retains 98.69%, while the two MLP-2048 fits retain 89.03% and 93.85% at $N = 512$ and $N = 2048$. The width-512 endpoints are slower. Every speculative method scores 105/128 with seven cap-length outputs. AR scores 104/128 with eleven. All speculative methods have identical per-question correctness in this sample, but the conservative paired 95% difference interval is still $[-3.37, 3.37]$ percentage points. Against AR, dense $N = 2048$ has four beneficial and three adverse correctness discordances. Equal or close scores therefore do not establish tight noninferiority or uncapped answer quality. The observed small-data retention replicates across the two drafter families under these development conditions, not across independent benchmark samples.

C.5 Input-only task complexity

We stratify the same 128 development requests using the benchmark's original difficulty levels and subjects, together with shared tokenized input length. Group definitions are fixed before computing subgroup outcomes. They do not use candidate success, generated output length or latency. This is a retrospective analysis of exposed development data, not a new confirmatory test. All twelve methods and all requests remain in the analysis.

Table 39: Throughput retained from dense $N = 2048$ within each MATH difficulty group, in percent with descriptive paired 95% intervals. All fits use 8,192 updates except the continued width-512 fits, which use 32,768 updates on the same 2,048 examples. For fitted maps, unspecified N is 2,048. Intervals do not adjust for multiple subgroup comparisons or model selection.

Setting	Levels 1–2 ($n = 42$)	Level 3 ($n = 21$)	Levels 4–5 ($n = 65$)
AR	20.1 [18.6, 21.5]	19.0 [16.5, 22.1]	18.7 [17.8, 19.7]
Native drafter	101.4 [99.1, 104.1]	104.2 [102.0, 106.6]	103.7 [102.8, 104.6]
Source reuse	60.6 [59.2, 61.8]	60.7 [59.2, 62.1]	61.3 [60.6, 61.9]
Dense, $N = 512$	97.4 [96.1, 98.6]	97.9 [96.1, 100.2]	97.9 [97.0, 98.8]
Dense, $N = 2048$	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Linear 1024, $N = 512$	94.5 [93.2, 95.9]	94.8 [93.4, 96.5]	95.2 [94.0, 96.4]
Linear 4096, $N = 2048$	98.9 [97.9, 99.8]	100.0 [98.6, 101.5]	98.9 [98.2, 99.6]
MLP 4096, $N = 2048$	95.5 [94.3, 96.6]	96.8 [94.9, 98.7]	96.0 [95.1, 96.9]
Linear 512	82.6 [79.3, 85.7]	84.5 [81.6, 86.8]	83.0 [81.5, 84.5]
Linear 512, continued	84.1 [80.6, 87.5]	86.2 [84.0, 88.2]	84.2 [82.7, 85.7]
MLP 512	76.5 [73.4, 79.5]	79.9 [76.8, 82.5]	78.0 [76.2, 79.7]
MLP 512, continued	78.4 [74.6, 82.0]	79.8 [77.0, 82.0]	80.0 [78.1, 81.7]

Dense $N = 512$ retains 97.4–97.9% of dense $N = 2048$ throughput across the three difficulty groups. The width-1024 linear map stays near the 95% boundary, while MLP-4096 retains 95.4–96.8%. The smaller width-512 maps remain slower in every group. These results describe the tested capacities and workloads. They do not establish an optimal size across settings.

For levels 1–2, level 3 and levels 4–5, respectively, AR answers 38/42, 19/21 and 47/65 questions correctly. Every mapper answers 38/42, 18/21 and 49/65 correctly. All eleven AR cap hits occur in the highest difficulty group, as do eight of the nine mapper cap hits. Difficulty therefore coincides with changes in capped answer quality and output length. It is not a controlled intervention on decoding cost.

Input-length bins of at most 64, 65–128, 129–256 and more than 256 tokens contain 52, 57, 15 and 4 requests. In the final four-request group, dense $N = 512$ retains 94.5% [91.3, 96.4]% and linear-1024 retains 90.8% [88.2, 94.4]% of dense $N = 2048$ throughput. The small sample and multiple comparisons make this a signal for replication, not evidence of a causal length effect or a new tuning rule. All seven original subject strata, input assignments, method outcomes and conservative paired accuracy intervals are retained in the accompanying artifacts. No additional generation or fitting is used for this analysis.

C.6 CAPACITY REPLICATION WITH A 14B TARGET

We repeat the fixed-data capacity comparison with Qwen3-14B for both DFlash and EAGLE-3. Each family has fourteen primary fits: dense, factored linear and GELU MLP widths 512, 1,024 and 4,096 at each of $N = 512$ and $N = 2048$. Two additional dense fits use seed 1730. All receive 8,192 batch-four updates at learning rate 6×10^{-4}, with no L2 penalty or weight decay. Validation uses the same 1,024 records within each family. Training diagnostics use a 256-record prefix. The five target taps give 25,600 input dimensions and the output has 2,560 dimensions. Dense therefore has 65.54M parameters, versus 14.42M, 28.84M and 115.34M at the three matched factorized/MLP widths. Width 4,096 does not increase the maximum rank of a linear function beyond the output dimension.

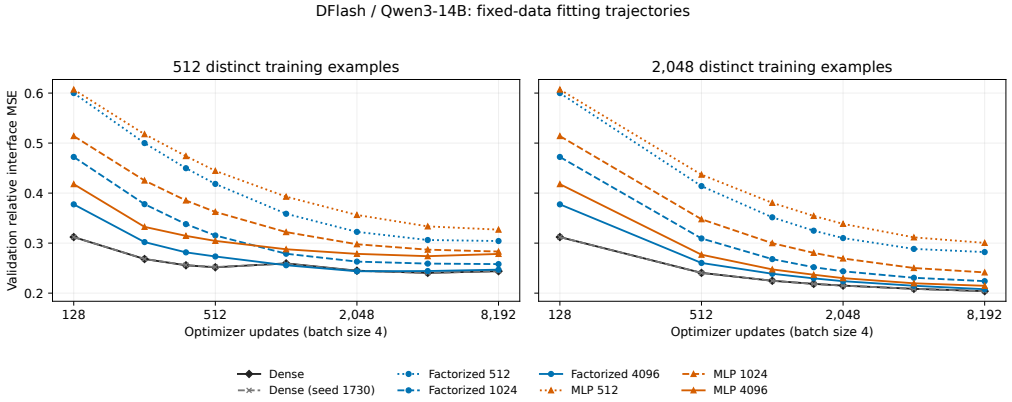

Figure 12: DFlash with Qwen3-14B. Validation error across the saved training trajectory, with all capacities and both dense seeds retained. The decoding endpoint remains fixed at 8,192 updates.

Table 40: DFlash-14B fixed endpoints. Throughput retention uses dense $N = 2048$, seed 1729. Intervals are paired 95% request-bootstrap intervals on sixteen exposed questions capped at 256 output tokens. They exclude fitting-seed and selection uncertainty.

Mapper	N	Params (M)	Train / val.	Tokens/s	Retained (%)
Dense	512	65.54	0.153 / 0.244	123.38	96.0 [92.3, 99.3]
Dense (seed 1730)	512	65.54	0.153 / 0.244	122.08	94.9 [91.5, 98.1]
Linear 512	512	14.42	0.251 / 0.304	99.16	77.1 [72.6, 81.8]
Linear 1024	512	28.84	0.180 / 0.258	117.39	91.3 [87.7, 95.1]
Linear 4096	512	115.34	0.155 / 0.247	121.65	94.6 [91.4, 97.9]
MLP 512	512	14.42	0.277 / 0.327	92.11	71.6 [68.1, 75.8]
MLP 1024	512	28.84	0.208 / 0.283	109.02	84.8 [79.7, 90.3]
MLP 4096	512	115.34	0.140 / 0.278	112.85	87.8 [83.5, 92.0]
Dense	2048	65.54	0.188 / 0.204	128.58	100.0 [100.0, 100.0]
Dense (seed 1730)	2048	65.54	0.188 / 0.204	129.55	100.8 [99.5, 102.1]
Linear 512	2048	14.42	0.274 / 0.282	105.53	82.1 [76.9, 87.4]
Linear 1024	2048	28.84	0.212 / 0.224	127.67	99.3 [96.5, 102.3]
Linear 4096	2048	115.34	0.193 / 0.208	126.96	98.7 [96.5, 101.2]
MLP 512	2048	14.42	0.292 / 0.300	96.46	75.0 [70.9, 79.2]
MLP 1024	2048	28.84	0.227 / 0.241	121.21	94.3 [90.9, 98.1]
MLP 4096	2048	115.34	0.174 / 0.215	122.32	95.1 [92.6, 98.2]

With DFlash, the wide MLP at $N = 2048$ has lower training error than dense, but higher validation error and lower measured throughput. The width-1024 linear map nearly matches dense throughput with fewer parameters. These comparisons use fixed endpoints, not checkpoints chosen after inspecting decoding. Every method reaches the output cap on these sixteen questions, so the short scores do not establish full-answer quality. The runtime includes AR and source-reuse controls. No native 14B DFlash checkpoint is substituted from another target.

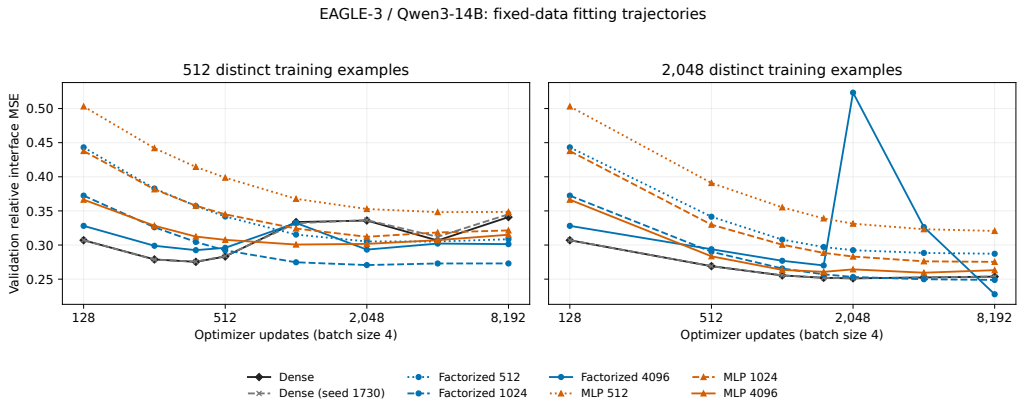

Figure 13: EAGLE-3 with Qwen3-14B under the same data-count and update budgets. The width-4096 linear trajectory retains its intermediate validation spike. Curves show optimization behavior, not a ranking of decoding performance or a selected stopping rule.

Table 41: EAGLE-3-14B fixed endpoints under its own runtime. Retention and intervals use the same within-family protocol as Table 40. Absolute throughput across the two tables is not an algorithm-only comparison.

Mapper	N	Params (M)	Train / val.	Tokens/s	Retained (%)
Dense	512	65.54	0.234 / 0.341	62.24	94.4 [91.7, 97.2]
Dense (seed 1730)	512	65.54	0.238 / 0.345	63.32	96.1 [92.7, 99.5]
Linear 512	512	14.42	0.256 / 0.308	56.92	86.4 [83.5, 89.4]
Linear 1024	512	28.84	0.197 / 0.273	62.50	94.8 [91.9, 98.0]
Linear 4096	512	115.34	0.226 / 0.301	60.90	92.4 [89.8, 95.1]
MLP 512	512	14.42	0.302 / 0.349	53.15	80.6 [77.4, 84.1]
MLP 1024	512	28.84	0.253 / 0.321	59.45	90.2 [87.3, 92.8]
MLP 4096	512	115.34	0.224 / 0.315	52.64	79.9 [76.6, 83.5]
Dense	2048	65.54	0.231 / 0.254	65.91	100.0 [100.0, 100.0]
Dense (seed 1730)	2048	65.54	0.231 / 0.254	66.45	100.8 [99.3, 102.4]
Linear 512	2048	14.42	0.279 / 0.287	58.92	89.4 [86.0, 92.7]
Linear 1024	2048	28.84	0.236 / 0.249	63.41	96.2 [94.6, 97.8]
Linear 4096	2048	115.34	0.216 / 0.228	65.61	99.5 [97.5, 101.9]
MLP 512	2048	14.42	0.311 / 0.321	56.54	85.8 [82.2, 89.3]
MLP 1024	2048	28.84	0.259 / 0.275	61.76	93.7 [90.5, 97.0]
MLP 4096	2048	115.34	0.227 / 0.263	64.02	97.1 [94.9, 99.4]

EAGLE-3 differs from DFlash in its fitting dynamics. At $N = 512$, both dense seeds have their lowest saved validation error at 384 updates, followed by substantial increases. At $N = 2048$, the wide linear endpoint has lower validation error than dense, but nearly equal decoding throughput. Its intermediate spike also shows that the shared learning rate does not produce uniformly smooth training. These observations motivate matched learning-rate checks before attributing a difference to the mapper function class. Four lower-rate 16-update pilots passed numerical and decoding checks, but their proper 8,192-update fits were not executed within the experiment timebox. The pilots do not establish whether a lower rate resolves the degradation. Lower feature error alone does not establish a decoding gain. Full-answer quality, broader workloads and additional fitting seeds remain separate from this capped comparison.

C.7 Matched warm-training budgets on 512 examples

We compare feature regression, connector cross-entropy (CE) and rank-32 drafter LoRA on the same 512-example pool. CE updates the connector while LoRA holds the initial connector fixed. Both use target-greedy teacher labels on the last 15 proposal positions. An equal three-rate development screen selects learning rate 2×10^{-5} for each adaptation family. Feature regression uses its declared rate 6×10^{-4}. The two LoRA seeds vary the update initialization and share the same initial mapper. They do not measure variation across initial mapper fits.

We measure 91.413 seconds of optimizer-update time for the 8,192-update feature baseline and declare budgets of 0.25, 1 and 4 times that cost. CE and LoRA begin with a 128-update mapper, whose measured 1.546 seconds of training counts toward each budget. Each worker stops after the first completed update reaching its remaining budget. Recorded overshoot is at most one update, ranging from 0.002 to 0.201 seconds across the twelve fits. The three four-GPU fitting jobs finish in 2m33s, 2m35s and 7m12s.

Table 42: Measured training costs and subsequent decoding in one common development campaign. Worker time includes model/cache setup, validation and export where performed, plus the measured initial mapper worker cost for CE and LoRA. It excludes Python startup and shared cache construction. Ratios compare against feature regression at the same warm budget.

Budget	Objective	Updates	Warm s	Worker s	Tok/s	Feature ratio [95% CI]
0.25×	Feature MSE	2026	22.86	52.04	145.05	1.000 [1.000, 1.000]
0.25×	Connector CE	106	22.94	91.89	111.04	0.766 [0.737, 0.791]
0.25×	LoRA32 (1729)	95	23.00	94.66	119.30	0.823 [0.786, 0.859]
0.25×	LoRA32 (1730)	94	22.86	95.20	119.52	0.824 [0.788, 0.860]
1×	Feature MSE	8144	91.42	119.71	144.80	1.000 [1.000, 1.000]
1×	Connector CE	474	91.58	161.04	109.60	0.757 [0.726, 0.786]
1×	LoRA32 (1729)	402	91.45	165.73	115.02	0.794 [0.757, 0.831]
1×	LoRA32 (1730)	407	91.50	164.61	115.49	0.798 [0.765, 0.830]
4×	Feature MSE	32446	365.65	390.11	144.55	1.000 [1.000, 1.000]
4×	Connector CE	1899	365.72	435.56	105.79	0.732 [0.699, 0.763]
4×	LoRA32 (1729)	1651	365.85	438.51	118.49	0.820 [0.786, 0.853]
4×	LoRA32 (1730)	1646	365.76	437.99	119.55	0.827 [0.793, 0.860]

Table 43: References measured within the same budget-decoding campaign. The initial mapper has 128 feature-regression updates. The 8,192-update reference is an existing saved checkpoint.

Reference	Tok/s	Initial mapper ratio [95% CI]
AR	28.61	0.240 [0.207, 0.277]
Native drafter	151.71	1.275 [1.205, 1.348]
Source reuse	84.71	0.712 [0.684, 0.737]
Initial mapper	119.01	1.000 [1.000, 1.000]
Dense, 8192 updates	145.82	1.225 [1.154, 1.302]

Feature regression reaches 145.05, 144.80 and 144.55 tokens/s at the three budgets. Relative to the quarter-budget fit, the 1-times and 4-times fits retain 99.83% [97.18, 102.53]% and 99.66% [97.35, 102.11]%, respectively. These intervals do not establish a benefit from longer fitting. Connector CE retains 73.2–76.6% of same-budget feature throughput, and the two LoRA seeds retain 79.4–82.7%. This result concerns the declared teacher-forced objective, initialization, rank and development-selected learning rates. It does not establish that other adaptation objectives or full drafter training fail.

Warm training is a marginal cost with cached features available. The historical full-cache extraction took 712.58 seconds and produced 292.8 GB. We reuse its first 512 training records and report the original construction cost separately. Dividing that cost by the record count would not measure a fresh 512-example extraction. The timed fits do not add training examples or repeat large-data scaling.

All seventeen methods decode the same sixteen exposed development questions with a 256-token cap. Tables 42 and 43 report descriptive paired request bootstrap intervals. They do not provide full-answer quality, untouched confirmation or corrections for selecting among multiple comparisons. Absolute throughput is interpreted within this common campaign and its deterministic training/runtime configuration.

C.8 Draft length and interface choices

The block-size experiment directly compares AR, native DFlash and the relay at lengths 8, 16 and 32 on 64 fixed requests. Block 16 gives the highest relay throughput, 210.15 tokens/s (Table 13). Block 8 retains more source progress proportionally but advances fewer tokens per cycle. Block 32 adds verification width while reducing accepted progress for the released block-16 drafter. This supports keeping its trained block length instead of maximizing the relative gain over source reuse.

The EAGLE-3 normalization comparison holds parameter count, data and 128-update budget fixed. Raw target features reduce relative context error from 0.413 to 0.266 and increase the source-relative throughput ratio from 1.035 to 1.237. The raw interface consumes feature magnitude, which input normalization removes. This gives a concrete reason for preserving scale in the selected map.

Appendix C reports the DFlash objective, ridge and normalization comparisons with their distinct fitting recipes.

D External baselines in their public runtimes

Table 44: Mechanism and evaluation scope of the closest alternatives. All rows inherit pretrained models. "No new fit" for PARD refers to retargeting its released checkpoint, not its original training cost. DFlash and EAGLE-3 are the inherited drafter families rather than independent competitors to RelaySpec's interface replacement.

Method	New-target adaptation	Use of target features	Evidence here
RelaySpec	Fit input map, freeze drafter	Replace source conditioning	Two drafter families, 8B/14B targets
PARD (An et al., 2026)	No new fit for released drafter	Target-independent proposals	Public runtime, 128-question capped quality
SD^2 (Berdoz et al., 2026)	Fit steering in tested frozen-drafter option	Inject steering within drafter	Rate and epoch sweeps, 128-question capped quality
Connector CE	Fit input map with token CE	Same replacement interface	Common-runtime matched warm budgets
Drafter LoRA	Fit initial map, then LoRA with map frozen	Adapt feature-conditioned drafter	Common-runtime matched warm budgets, two seeds

We evaluate SD^2's public frozen-drafter option and the released PARD parallel drafter on sixteen exposed MATH development requests with a 256-token cap. Each system has a correct cached AR control using its own target runtime and identical pretokenized inputs. These are short development comparisons. Below we separately evaluate capped answer quality. No untouched confirmation is included in these measurements.

SD^2 trains 402,665,472 steering parameters with merged guidance from target layers 3, 16 and 33 into all drafter layers. Inherited target and drafter weights remain frozen. Each of six objective/rate settings sees 512 distinct records once, using 128 batch-four updates and 95,835 non-padding supervised positions. Records are capped at 192 tokens. Training takes 47.0–48.6 seconds per fit. The initial TVD and KL settings also have exact fitting replicas, whose costs remain in the experiment record. This supervision differs from the feature-regression and last-15-position adaptation objectives in the preceding study.

Table 45: One-epoch SD^2 development comparison. Intervals resample paired requests 10,000 times and do not adjust for rate selection. Exact AR counts identical capped token sequences.

Setting	Tok/s	AR ratio [95% CI]	Independent ratio [95% CI]	Exact AR
Matched AR	12.64	1.000 [1.000, 1.000]	0.671 [0.622, 0.719]	16/16
Independent drafter	18.83	1.490 [1.391, 1.607]	1.000 [1.000, 1.000]	6/16
Zero guidance	18.56	1.469 [1.371, 1.584]	0.985 [0.983, 0.988]	6/16
TVD, 10^{-5}	18.38	1.454 [1.360, 1.567]	0.976 [0.955, 0.997]	4/16
KL, 10^{-5}	18.17	1.438 [1.341, 1.552]	0.965 [0.937, 0.993]	9/16
TVD, 4×10^{-6}	18.52	1.466 [1.370, 1.574]	0.984 [0.961, 1.006]	6/16
TVD, 2×10^{-5}	18.47	1.462 [1.365, 1.570]	0.981 [0.953, 1.011]	6/16
KL, 4×10^{-6}	18.66	1.477 [1.384, 1.588]	0.991 [0.962, 1.016]	5/16
KL, 2×10^{-5}	18.42	1.458 [1.363, 1.569]	0.978 [0.955, 1.003]	10/16

The public SD^2 evaluation configuration uses FP16 target storage, BF16 drafter and steering storage, and BF16 autocast under Transformers 4.52.4. We verify original FP32 training weights before conversion and separately fingerprint the BF16 inference tensors. The best fitted point, KL at 4×10^{-6}, retains 99.11% [96.19, 101.62]% of independent-drafter throughput. This screen does not establish a steering gain. We freeze this rate before checking additional epochs on the same small pool.

Table 46: Released PARD with zero new-target fitting, Transformers 4.51.3, BF16 weights, eager attention and static caches. The released checkpoint carries inherited proposer training.

Setting	Tok/s	AR ratio [95% CI]	Exact AR
Matched eager AR	31.27	1.000 [1.000, 1.000]	16/16
Released PARD	98.60	3.153 [2.831, 3.558]	4/16

Every SD^2 committed token matches its actual verifier decision, and immediate versus deferred observation preserves tokens and acceptance. Each timed PARD request has a separate verifier-observed replica with identical raw output and acceptance counts. PARD's measured generation omits that added observer. Both timers include prefills and cache setup, preserve raw EOS/cap overshoot and exclude detokenization. Earlier failed exact-AR gates remain failed. Controlled cache, query and attention replays expose changes in rounded target scores, while future-query perturbations preserve earlier logits in the tested calls. The sequence differences in these tables require separate task-quality evidence. Absolute speeds across these public runtimes do not isolate algorithm effects, and shared model residency does not measure isolated serving memory.

D.1 Additional epochs and capped answer quality

After the six-rate screen, we fix KL and the selected learning rate and fit for 1, 2, 4 and 8 epochs on the same 512 examples. These are four fresh fits, each with its own learning-rate decay horizon, rather than checkpoints from one training trajectory. The one-epoch fit exactly reproduces the selected earlier training fingerprint. All four endpoints are evaluated together on the same sixteen development requests, with the public SD^2 runtime and matched controls.

Table 47: SD^2 epoch budget at fixed 512 distinct examples. Training KL is the mean over updates in each final epoch. It is not held-out loss. Independent-drafter throughput is 18.75 tokens/s. Intervals resample paired requests and do not adjust for model selection.

Epochs	Fit seconds	Final-epoch training KL	Tok/s	Independent ratio [95% CI]
1	48.44	0.3923	18.56	0.9900 [0.9607, 1.0149]
2	97.01	0.2787	18.45	0.9841 [0.9639, 1.0033]
4	193.81	0.2027	18.39	0.9808 [0.9555, 1.0039]
8	388.08	0.1254	18.26	0.9738 [0.9488, 0.9997]

Training KL decreases from 0.3923 to 0.1254, while every fitted endpoint has a throughput point below the independent drafter. Additional epochs do not establish a decoding improvement in this fixed-pool setting. The highest-throughput fitted endpoint remains one epoch. We freeze this checkpoint before longer quality evaluation. This result concerns the tested frozen-drafter configuration, supervision and pool, and does not establish a general failure of SD^2 or its other training options.

Table 48: PARD capped answer quality on 128 exposed MATH development requests, after an eight-request pilot passed. Both methods use the same 2048-token cap and public BF16 eager runtime. Cap hits include all outputs reaching the cap. None ends in EOS exactly at the cap.

Setting	Tok/s	AR ratio [95% CI]	Correct	Cap hits
Matched eager AR	31.67	1.000 [1.000, 1.000]	103/128	11
Released PARD	105.63	3.335 [3.222, 3.451]	105/128	8

PARD has five beneficial and three adverse correctness discordances against AR, a difference of $+1.56$ percentage points. A conservative paired 95% interval is $[-6.34, 9.30]$ points: form exact 97.5% Clopper–Pearson intervals for each discordance probability and combine their opposite endpoints using a Bonferroni bound. This construction requires IID request pairs and a fixed candidate. It does not establish a one-percentage-point noninferiority margin. The descriptive paired bootstrap interval is $[-2.34, 5.47]$ points. Sparse or absent discordances can make such intervals misleadingly narrow. These external-baseline measurements use development questions.

PARD matches AR's full capped token sequence on 28 of 128 requests. Its mean output length is 664.5 tokens versus AR's 690.4, so the 3.34-fold token-throughput ratio differs from the 3.47-fold request-time ratio. Every measured PARD request retains a separate, exactly matching verifier-observed replica. These are development outcomes under the specified runtime and scorer, not untouched confirmation or an isolated algorithm ranking across runtimes.

Table 49: Selected one-epoch SD^2 on the same 128 exposed MATH development requests, with a 2048-token cap. The public mixed-precision runtime and matched AR control are retained. Both have 11 cap-length outputs, none with EOS exactly at the cap.

Setting	Tok/s	AR ratio [95% CI]	Correct	Cap hits
Matched AR	12.48	1.000 [1.000, 1.000]	103/128	11
Selected SD^2	19.92	1.596 [1.560, 1.632]	103/128	11

SD^2 and matched AR both score 103 of 128, with three beneficial and three adverse correctness discordances. The conservative paired 95% interval is $[-7.04, 7.04]$ percentage points, which does not establish the one-point noninferiority margin. The descriptive paired bootstrap interval is $[-3.91, 3.91]$ points. Full capped sequences agree on 42 requests. Mean output lengths are 679.1 versus 680.9 tokens, so the 1.60-fold token-throughput and request-time ratios are close. Both shards retain the same selected training and inference fingerprints, and saved answers reproduce under the pinned scorer.

The public physical-buffer limit can end an untimed warmup before its token cap. We record this termination only during warmup. Measured requests must still reach EOS or the cap and pass all actual-verifier checks. A replacement eight-request pilot passed before the complete quality run. Failed preliminary attempts remain excluded from the reported timings. These results concern the tested frozen-drafter configuration and its public runtime, not all SD^2 training options.

E ACCEPTANCE AND NUMERICAL AGREEMENT

A retention value of 90% means the reused drafter produces 90% as many tokens per second as the target's own drafter. It is not the fraction of speedup above AR. We also report the ratio of summed AR request time to summed relay request time. This differs from a throughput ratio when output lengths differ.

Acceptance explains how speed is obtained. We report the pooled number of tokens advanced per proposal-verification cycle, including the implementation's target-supplied progress. This is distinct from the fraction of proposed draft tokens accepted. Appendix E gives the position-wise curves and precise counting convention.

Let c_R be the average relay cycle time and a_R its average progress. Its decode time per output token is approximately c_R/a_R. If c_A is AR time per token, then the decode-only approximation is

$$\text{AR-relative decode speed} \approx \frac{a_R c_A}{c_R}. \tag{4}$$

Removing source conditioning reduces cycle cost. Imperfect feature matching may reduce progress. The map is useful when the saved computation outweighs the extra cycles. We measure end-to-end speed directly and use this approximation to interpret the component measurements.

We pool all recorded proposal cycles rather than averaging each request's mean. This weights cycles equally. DFlash's recorded length is one already target-selected position plus the accepted draft prefix. EAGLE-3's length follows its shared decoder's progress count. These quantities measure work advanced per cycle, including target-supplied progress. A proposed-token acceptance fraction requires separating those positions and accounting for the number proposed.

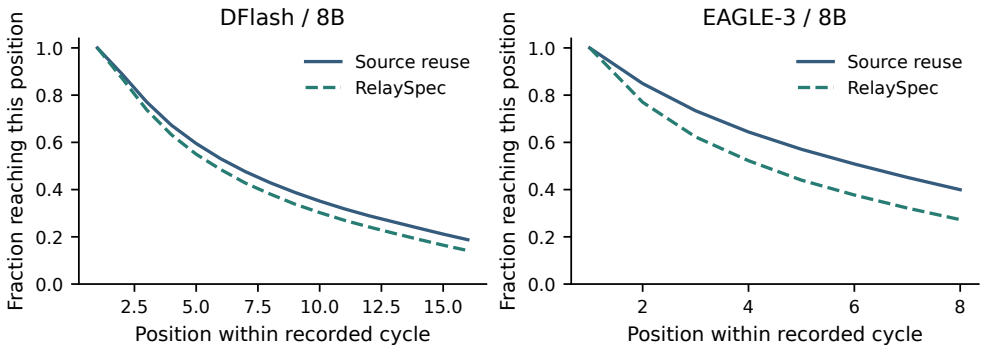

Figure 14: Historical 8B source-reuse comparisons. Each curve gives the fraction of recorded cycles reaching each position. Summing the curve values gives mean recorded progress per cycle. Solid and dashed lines show source reuse and RelaySpec, respectively.

The main MATH-500 table reports measured full-sequence agreement with AR. A fresh diagnostic selects eight prompts with early differences and saves target inputs, cache lengths, positions and the two leading logits at every call. The DFlash paths share the same first differing choice across native, source and relay drafting. The observed BF16 logits include ties or changed ordering with gaps up to 0.25 at those positions. Such traces identify numerical score differences in the selected cases. They do not establish that all mismatches have the same cause. The task-quality intervals in Table 2 remain the empirical quality evidence.

F MATCHED-COUNT TRAINING COMPOSITION

We compare two 2,048-record DFlash-8B fitting pools. The math-only arm uses NuminaMath records. The mixed arm replaces half with 1,024 Dolly general-instruction records.[2] Exact and lexical filters exclude matches to fitting validation, recorded evaluation prompts and the confirmation and reserve sets. The common validation set contains 1,024 math and 1,024 general-instruction records. Its cached feature files are identical across arms.

Each arm fits dense, linear-512 and MLP-512 maps at seed 1729, plus a second dense seed, 1730. Each fit uses 8,192 batch-four updates, learning rate 0.0006, no regularization and the same relative

[2] `databricks/databricks-dolly-15k`, revision `bdd27f4d94b9`, CC-BY-SA 3.0. General instructions can include mathematical questions.

interface objective. We retain the 2,048-update checkpoint as a fitting diagnostic, but preselect the 8,192-update endpoint for decoding rather than selecting by validation. Four independent fits run simultaneously on four L40S GPUs. Feature extraction took 111.4 seconds for the math cache and 107.1 for the mixed cache. The full fitting allocations took 4m36s and 4m21s, respectively. Extraction and fitting are separate costs.

Equal record counts and updates do not imply equal token exposure. With a 192-token record cap, the math pool contains 385,492 non-padding tokens and the mixed pool 316,518. Sixteen passes expose 6,167,872 and 5,064,288 tokens, respectively. The experiment therefore measures composition at matched records and updates, not matched tokens.

Table 50: Composition fitting at the preselected 8,192-update endpoint. Validation columns use the same held-out features in both arms. Reported seconds cover per-worker data access, updates and training-log writes, excluding validation, export and setup. Training losses concern different input distributions and are not directly comparable across arms.

Mapper	Training	Train loss	Math val.	General val.	Fit seconds
Dense	math	0.1695	0.1805	0.3329	177.0
Linear-512	math	0.2629	0.2685	0.4719	114.5
MLP-512	math	0.2812	0.2878	0.4928	110.3
Dense (seed 2)	math	0.1695	0.1805	0.3328	173.6
Dense	mixed	0.1854	0.1990	0.2234	167.0
Linear-512	mixed	0.3027	0.3015	0.3293	97.1
MLP-512	mixed	0.3221	0.3233	0.3479	110.2
Dense (seed 2)	mixed	0.1852	0.1990	0.2231	152.7

Mixed training lowers dense general-instruction validation error by 32.9%, from 0.3329 to 0.2234, while raising math error by 10.2%, from 0.1805 to 0.1990. The second dense seed reproduces this tradeoff. At width 512, the linear and MLP maps show general-error reductions of 30.2% and 29.4%, with math-error increases of 12.3% in both cases. The matched linear map retains lower validation error than the MLP in both domains and both training arms. All three architectures improve in both validation domains between 2,048 and 8,192 updates.

Fixed cross-domain decoding protocol. Before observing endpoint results, we declare 32 development inputs from each of MATH-500, GSM8K, HumanEval and MT-Bench. The 128 initial inputs expand to 160 timed requests because MT-Bench includes two turns. All eight fixed endpoints run alongside AR, native DFlash and source reuse, producing 1,760 measured outputs capped at 2,048 tokens. A separate eight-input operational pilot completes in 2m58s before this full run. Methods rotate within requests in the same runtime. Ratios compare mixed and math-only fits of the same architecture and seed. Paired bootstrap intervals resample whole conversations for MT-Bench and individual requests elsewhere. They describe this development sample, without multiplicity adjustment or fitting-seed uncertainty.

The feature-domain tradeoff carries through to decoding, but depends on mapper capacity and task. Mixed dense training changes throughput by -1.2% on MATH, $+1.6\%$ on GSM8K, $+2.9\%$ on HumanEval and $+4.9\%$ on MT-Bench. The second dense seed gives -1.7%, $+1.2\%$, $+2.4\%$ and $+4.7\%$, respectively. The first seed's MATH interval includes no change. Linear-512 and MLP-512 gain 11.6% and 10.8% on chat, but lose 6.2% and 7.8% on MATH and 2.6% and 4.1% on GSM8K. Dense remains faster than either smaller map on every task in both arms. General-domain feature improvement therefore supports useful workload-specific gains, not a uniform advantage from mixed training.

Every matched math/mixed pair produces identical generated token sequences on all 160 requests, including conversation turns. Thus these measured throughput gains do not arise from shorter outputs. All eight maps score 22/32 on MATH and 30/32 on GSM8K. AR scores 22/32 and 31/32. Four MATH outputs per method reach the cap, and none elsewhere does. The conservative paired accuracy-difference interval between matched arms is $[-12.80, 12.80]$ percentage points on each math benchmark. Equal observed outcomes on this small sample do not establish tight population

quality noninferiority. Code and conversation outputs remain unscored for task quality. The full decoding allocation took 52m30s.

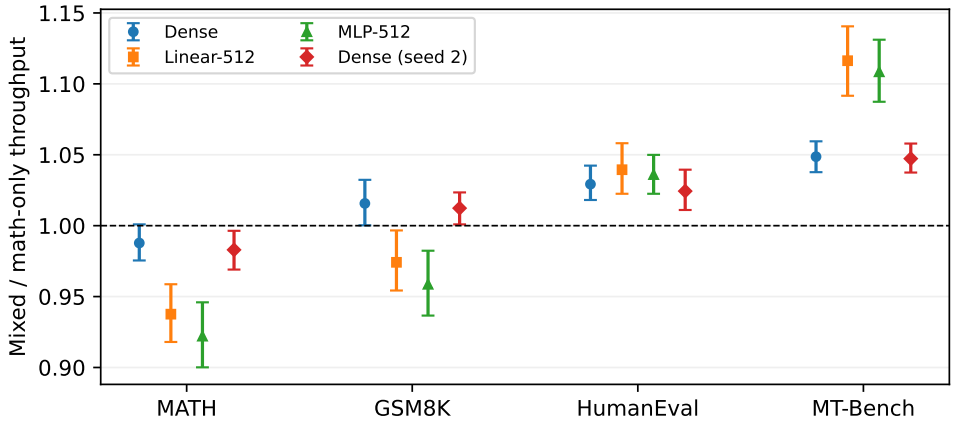

Figure 15: Effect of training composition on measured DFlash-8B throughput. Values above one favor mixed training. Marker shapes distinguish the four matched comparisons. Bars show paired 95% intervals. Code and conversation throughput does not establish output quality.

Table 51: All predeclared composition decoding comparisons. Throughput includes prompt processing. Ratios are mixed divided by math-only throughput, with paired 95% intervals. MT-Bench has 64 timed turns from 32 conversations. Each other task has 32 requests.

Task	Mapper	Math tokens/s	Mixed tokens/s	Ratio (95% CI)
MATH	Dense	201.1	198.6	0.988 [0.975,1.001]
GSM8K	Dense	156.5	159.0	1.016 [1.000,1.032]
HumanEval	Dense	138.1	142.2	1.029 [1.018,1.042]
MT-Bench	Dense	71.1	74.6	1.049 [1.038,1.059]
MATH	Linear-512	165.4	155.1	0.938 [0.918,0.959]
GSM8K	Linear-512	120.3	117.2	0.974 [0.954,0.997]
HumanEval	Linear-512	69.9	72.7	1.039 [1.022,1.058]
MT-Bench	Linear-512	49.1	54.8	1.116 [1.092,1.141]
MATH	MLP-512	155.6	143.5	0.922 [0.900,0.946]
GSM8K	MLP-512	112.7	108.0	0.959 [0.937,0.982]
HumanEval	MLP-512	64.4	66.7	1.036 [1.022,1.050]
MT-Bench	MLP-512	46.5	51.5	1.108 [1.087,1.131]
MATH	Dense (seed 2)	201.6	198.2	0.983 [0.969,0.996]
GSM8K	Dense (seed 2)	156.7	158.6	1.012 [1.001,1.023]
HumanEval	Dense (seed 2)	138.0	141.4	1.024 [1.011,1.039]
MT-Bench	Dense (seed 2)	71.1	74.5	1.047 [1.037,1.058]

Table 52: Composition evaluation: correct-answer and output-cap counts. MATH and GSM8K columns show correct answers followed by capped outputs, each out of 32. Code cap counts are out of 32 and chat cap counts out of 64 turns. HumanEval execution and MT-Bench answer quality are not scored in this experiment. Capped outputs remain in every aggregate.

Method	MATH correct / cap	GSM8K correct / cap	Code cap	Chat cap
AR	22 / 4	31 / 0	0	0
Native DFlash	22 / 4	30 / 0	0	0
Source reuse	22 / 4	30 / 0	0	0
Dense, math	22 / 4	30 / 0	0	0
Dense, mixed	22 / 4	30 / 0	0	0
Linear-512, math	22 / 4	30 / 0	0	0
Linear-512, mixed	22 / 4	30 / 0	0	0
MLP-512, math	22 / 4	30 / 0	0	0
MLP-512, mixed	22 / 4	30 / 0	0	0
Dense (seed 2), math	22 / 4	30 / 0	0	0
Dense (seed 2), mixed	22 / 4	30 / 0	0	0

F.1 Composition with compact interfaces

We separately screen two-layer interfaces using 512 math records or 256 math plus 256 general-instruction records, at 8,192 updates and seed 1729. Both use target layers [25,33]. Retargeted maps fit the source-drafter interface. Native students fit the released native projection and normalization. Thus this comparison changes calibration composition at fixed interface capacity and record count, but not necessarily equal token exposure.

The largest observed difference is on retargeted dialogue: mixed calibration retains 110.8% [107.8,112.5]% of math-only throughput. All seven other intervals include parity. Every paired output matches within these capped screens. This is a workload-specific exploratory finding from eight conversations and one fitting seed, not evidence that mixed calibration improves decoding across workloads or answer quality.

Table 53: Compact-interface calibration composition. Ratios compare both checkpoints within the same worker. Paired 95% bootstrap intervals use requests, or whole conversations for dialogue, and exclude fitting-seed and selection uncertainty. Code and math screens use 128-token caps. dialogue uses 256 tokens per turn.

Interface	Workload	Units	Mixed / math throughput (95% CI)
Native	Code	32	100.6% [98.6, 102.7]
Native	Dialogue	8	101.1% [98.8, 102.7]
Native	GSM8K	32	101.2% [99.6, 103.0]
Native	MATH	32	101.5% [99.4, 103.4]
Retargeted	Code	32	102.1% [98.8, 105.6]
Retargeted	Dialogue	8	110.8% [107.8, 112.5]
Retargeted	GSM8K	32	99.3% [96.8, 101.9]
Retargeted	MATH	32	101.1% [96.7, 105.5]

We then repeat fitting at seed 1730 and compare both seeds on sixteen additional development conversations, disjoint from the initial eight. Within each worker, all four compact maps are paired. Mixed/math throughput ratios are 107.50% [104.34,110.44]% at seed 1729 and 107.52% [104.44,110.21]% at seed 1730 for retargeting. Native students give 101.09% [100.12,102.08]% and 100.28% [99.47,101.14]%, respectively. All 32 paired turn outputs match at each seed. Both seeds share the same conversations, so they are fitting replications rather than independent benchmark samples. This extension remains development evidence, with the same 256-token cap and no dialogue-quality scoring.

The fitting diagnostics help delimit the result. At seed 1729, mixed calibration lowers general-instruction validation error from 0.445 to 0.289 for retargeting and from 0.0468 to 0.0298 for native students. Both improve substantially in relative terms, but only retargeting shows a sizeable dialogue

throughput gain. Math errors increase from 0.227 to 0.251 and from 0.0241 to 0.0267, respectively. These diagnostics use the same 64 validation records per domain. The pattern suggests that domain coverage matters more for the harder retargeted interface in these tests. It also shows why proportional feature-error improvements alone do not predict proportional decoding gains.

G FROZEN-CHECKPOINT GSM8K CONFIRMATION

We freeze the dense 512- and 2,048-example checkpoints at 8,192 updates and seed 1729 before generating confirmation outputs. The fixed test contains 256 GSM8K questions, sampled after exact and lexical exclusion against fitting data and collected evaluation history. A separate 256-question reserve was left unused by this experiment. Subsequently, 128 reserve questions were consumed by two separately frozen studies: 64 for the interface/SVD comparison and 64 different questions for native student confirmation (Appendix C). Operational pilots use eight previously exposed GSM8K questions, not confirmation requests. The same confirmation questions are evaluated in both drafter families, so the second family is not an independent benchmark sample.

The primary speed criterion requires the lower endpoint of the paired 95% throughput-ratio interval to exceed 0.95. DFlash retains 97.48% [96.90,98.06]% and EAGLE-3 retains 98.90% [98.14,99.66]%. Throughput includes prompt processing. Resampling uses requests as paired units, 10,000 draws and seed 1729. These intervals condition on the frozen fitting checkpoints rather than estimating fitting-seed variation.

Table 54: All frozen-confirmation methods on 256 GSM8K requests per family. At cap counts outputs of exactly 2,048 tokens. Comparisons are within each family's runtime.

Family	Method	Tokens/s	Correct	At cap
DFlash	AR	38.0	236/256	0
DFlash	Native drafter	170.3	236/256	0
DFlash	Source reuse	99.0	236/256	0
DFlash	Dense 512	157.8	236/256	0
DFlash	Dense 2,048	161.9	236/256	0
EAGLE-3	AR	36.9	236/256	0
EAGLE-3	Native drafter	100.9	238/256	0
EAGLE-3	Source reuse	70.7	238/256	0
EAGLE-3	Dense 512	100.0	238/256	0
EAGLE-3	Dense 2,048	101.1	238/256	0

The predeclared quality criterion requires the conservative paired 95% accuracy-difference lower bound relative to AR to exceed -1 percentage point. DFlash has two beneficial and two adverse discordances, giving $[-3.07, 3.07]$ points. EAGLE-3 has two beneficial and no adverse discordances, giving $[-1.63, 3.14]$ points. Both quality outcomes are inconclusive under this criterion. Therefore the joint speed-and-quality criterion is not established, despite both speed criteria passing. Within each family, the two mapper sizes have identical per-question correctness, with a conservative interval of ± 1.70 points. DFlash also has identical token outputs between mapper sizes on all 256 requests, while EAGLE-3 has identical outputs on 254. Comparisons with AR retain the finite-precision distinctions discussed elsewhere. No reserve extension or outcome-dependent tuning is performed. Exclusion checks concern the recorded local history and lexical matches, not unknown pretraining exposure or semantic independence.

H DATA CHECKS AND DEVELOPMENT EXPOSURE

The 4,096-record fitting manifest contains distinct problems with solutions. The loader renders both fields before applying the 192-token cap. Repeating the manifest changes presentations, not distinct records. The 32-question development set informed normalization and objective choices. The historical 468-question complement is computed from saved full-run outputs and retains that exposure history.

Normalized exact matching finds zero overlaps between the 4,096 fitting and 1,250 evaluation problem texts. For a second check, each text is split into unique normalized tokens. Token-set Jaccard overlap is the number of shared tokens divided by the number of tokens in either set. Each evaluation question is compared with its closest fitting record. Table 55 reports the resulting similarities.

Table 55: Closest fitting-to-evaluation token-set overlap. Threshold columns count evaluation records at or above the stated overlap.

Task	Maximum overlap	≥ 0.80	≥ 0.90	≥ 0.95
MATH-500	0.980	47	13	4
GSM8K	0.378	0	0	0
HumanEval	0.290	0	0	0
MBPP	0.333	0	0	0
MT-Bench	0.571	0	0	0

MATH-500 includes 47 questions above 0.80 overlap and four above 0.95. These shared templates qualify the interpretation of math generalization. The fixed-map code and conversation evaluations give additional task coverage without changing the fitted map.

I FITTING TIME AND ISOLATED MEMORY

Table 56: Selected map size and warm fitting-loop duration on four RTX 6000 Ada GPUs. Models are loaded before the recorded loop begins.

Drafter	Target	Trainable weights (millions)	Fitting loop (seconds)
DFlash	8B	52.43	85.6
DFlash	14B	65.54	118.0
EAGLE-3	8B	52.43	86.7
EAGLE-3	14B	65.54	118.5

The fitting-loop timer covers source and target feature computation and matrix optimization. A complete setup also loads model weights, writes the map, reloads it and initializes generation. Published original training counts in Table 58 provide context for the additional-data requirement. They are not a wall-time or token-budget comparison with fitting a new native drafter.

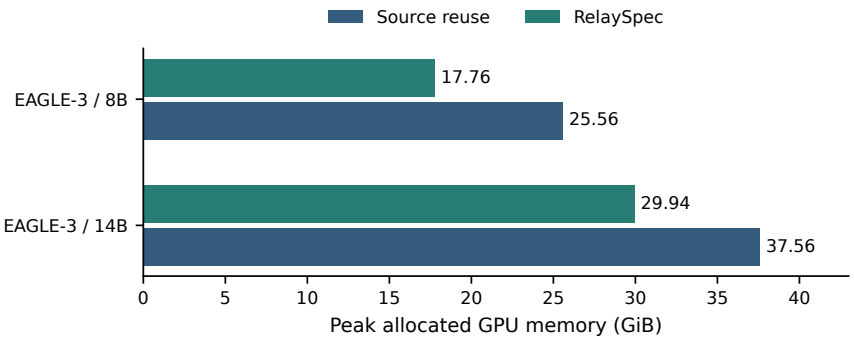

Figure 16: Isolated EAGLE-3 peak allocated memory on RTX 6000 Ada. Each method runs in a fresh process on eight paired prompts per target. Source removal saves 7.80 GiB at 8B and 7.63 GiB at 14B.

GPU peak counters are reset before the measured request. Allocated memory counts live tensor allocations and differs from reserved allocator memory. The mixed-arm processes used for timing are not the basis of this source-removal measurement.

I.1 SELECTIVE FEATURE CAPTURE DURING GENERATION

A small conditioning interface also makes it unnecessary to retain all verifier-layer outputs. We implement selective forward hooks for the requested target taps, preserving the final normalization for the last layer. The transformer, mapper weights, verifier decisions and KV-cache policy are unchanged. This is an implementation consequence of the interface, not a new compression algorithm. Even the five-tap mapper benefits: reducing the tap count is not required for the basic capture optimization.

On Qwen3-8B/DFlash, we compare the same frozen checkpoint with full hidden-state return and selective capture. Each method has two repetitions in reversed order on one constructed archive prompt per length, with a 64-token output cap. Every pair matches output token hashes, output lengths, acceptance trajectories and target/draft calls. At 31,526 input tokens, the two-tap mapper reduces peak allocated memory from 32.73 to 25.03 GiB (23.5%), and the five-tap mapper from 32.98 to 25.87 GiB. A stricter control immediately releases prefill outputs and concatenated features after conditioning in both methods: savings remain 7.70 and 7.11 GiB, respectively. In that control, paired request time ratios range from 0.994 to 1.002 across the two lengths and maps, so these runs show memory savings without a material runtime change.

Input tokens	Taps	Release prefill	All states (GiB)	Selected (GiB)	Saved (GiB)
2,273	5	No	18.35	17.83	0.52
2,273	2	No	18.22	17.66	0.56
8,993	5	No	21.70	19.67	2.03
8,993	2	No	21.54	19.35	2.20
16,826	5	No	25.62	21.83	3.80
16,826	2	No	25.43	21.33	4.11
31,526	5	No	32.98	25.87	7.11
31,526	2	No	32.73	25.03	7.70
16,826	5	Yes	25.62	21.83	3.80
16,826	2	Yes	25.43	21.32	4.11
31,526	5	Yes	32.98	25.87	7.11
31,526	2	Yes	32.73	25.03	7.70

Table 57: Selective-capture memory diagnostic on L40S. Values are mean peak allocated GiB over two order-reversed repetitions; these repeats are not independent prompts. Both frozen mapper copies are resident for both methods. Release-prefill rows control buffer lifetime after conditioning. Allocator-reserved memory is not the reported quantity.

A separate correctness check on eight code requests and eight two-turn dialogue conversations also preserves every output and acceptance trajectory for both the five-tap and two-tap mappers, with two order-reversed repetitions. These repeated executions do not add independent quality observations.

The synthetic inputs isolate sequence-length-dependent allocation and do not establish document understanding, long-context quality or concurrent-request capacity. Absolute values include model weights and the resident comparison maps. These results are distinct from the training-record scaling experiment and from the isolated source-removal experiment above.

J ADDITIONAL RELATED WORK

Improving drafting. EAGLE predicts future features with shifted tokens (Li et al., 2024a). EAGLE-3 combines feature layers and simulated drafting steps (Li et al., 2025). DFlash predicts a block in parallel (Chen et al., 2026), while Medusa uses several prediction heads (Cai et al., 2024). GRIFFIN addresses training-to-decoding token alignment (Hu et al., 2025). RepSpec changes the training structure (Huo et al., 2026), while VSD and Draft-OPD optimize accepted proposals (Zou et al., 2026; Lei et al., 2026). RelaySpec inherits the trained drafter and changes only its conditioning input.

Proposal structure and execution. EAGLE-2 and OPT-Tree adapt proposal trees (Li et al., 2024b; Wang et al., 2025a). SpecInfer organizes tree verification (Miao et al., 2024). Distributed speculative inference and speculative speculative decoding overlap computation (Timor et al., 2025b; Kumar et al., 2026). Draft & Verify skips target layers (Zhang et al., 2024), LayerSkip trains early exits

(Elhoushi et al., 2024), and Lookahead checks candidates without an auxiliary drafter (Fu et al., 2024). RelaySpec retains an external drafter and holds its released proposal structure fixed. We evaluate the cost and accepted progress of changing its feature source.

K INHERITED DRAFTER TRAINING CONTEXT

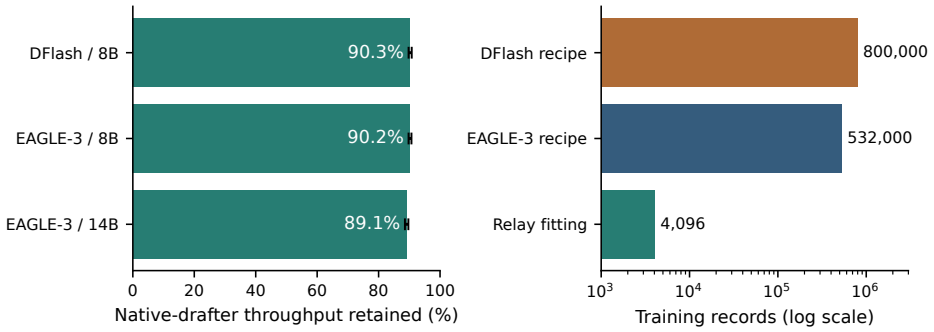

Figure 17: Left: fraction of native target-specific drafter throughput retained by RelaySpec in paired MATH-500 runs. Right: published original DFlash and EAGLE-3 recipe sizes versus the additional RelaySpec fitting set. Recipe counts provide training-scale context, not a matched training experiment or an audit of each released checkpoint.

Table 58: Original drafter training and additional target adaptation. Record counts describe different training stages and objectives.

Stage	Records	Relative to relay fit	Weights learned
Published DFlash recipe (Chen et al., 2026)	∼800,000	∼195×	Drafter
Published EAGLE-3 recipe (Li et al., 2025)	∼532,000	∼130×	Drafter
RelaySpec target adaptation	4,096	1×	Feature map

The DFlash recipe reports about 800K target-generated records. The EAGLE-3 recipe uses about 68K ShareGPT and 464K UltraChat entries. Relay fitting uses 0.51% and 0.77% of these respective record counts. This comparison describes marginal adaptation: the original drafter's training remains inherited. Record lengths, objectives and hardware differ, so these ratios do not represent equivalent reductions in training compute. The evaluated EAGLE-3 checkpoints come from DeepSpec, while the table's EAGLE-3 count refers specifically to the published training recipe.